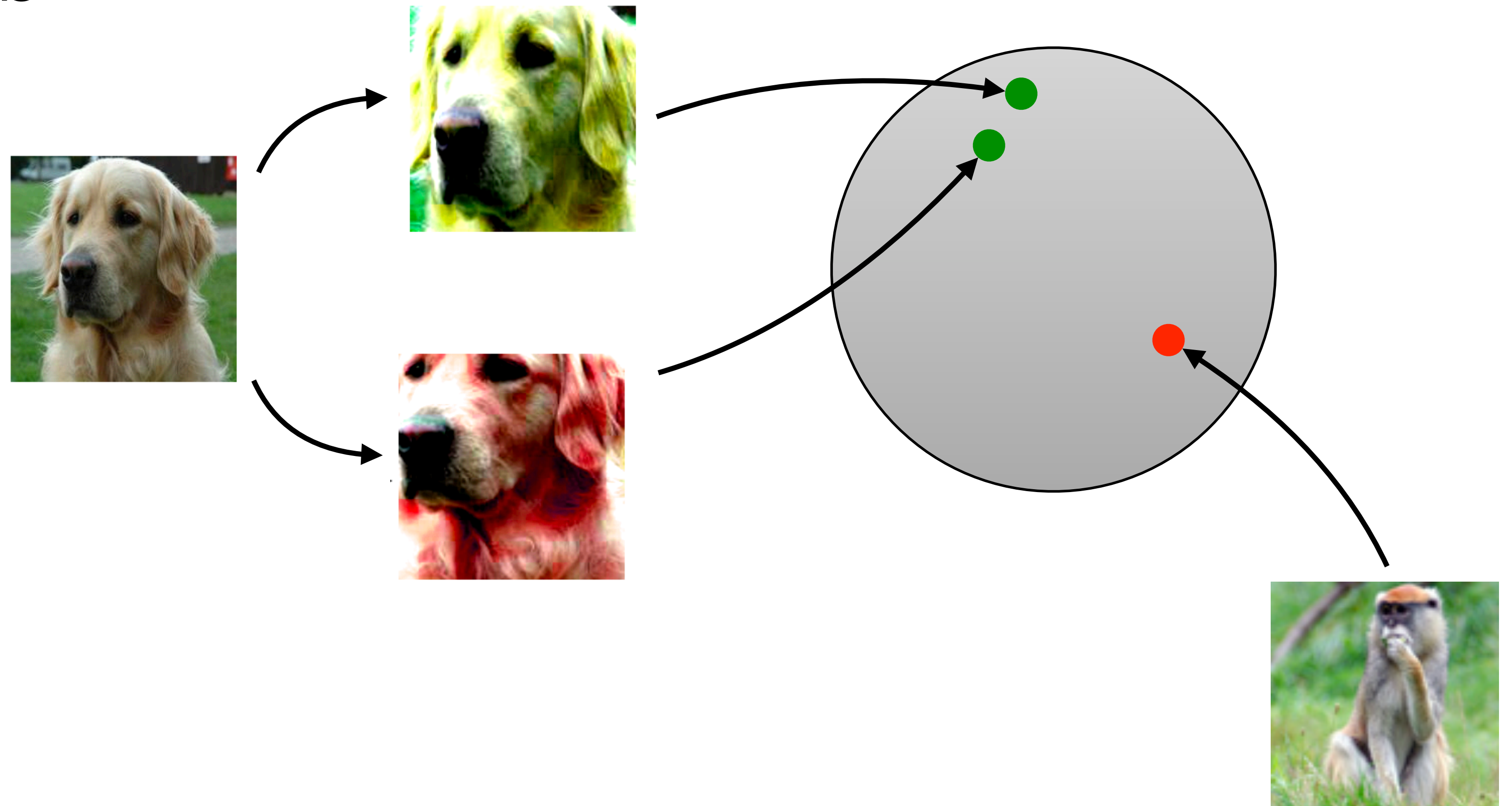


# Lecture 9: Similarity-based Representation Learning (Neural Information Retrieval)

Speaker: Omar Khattab



# Outline

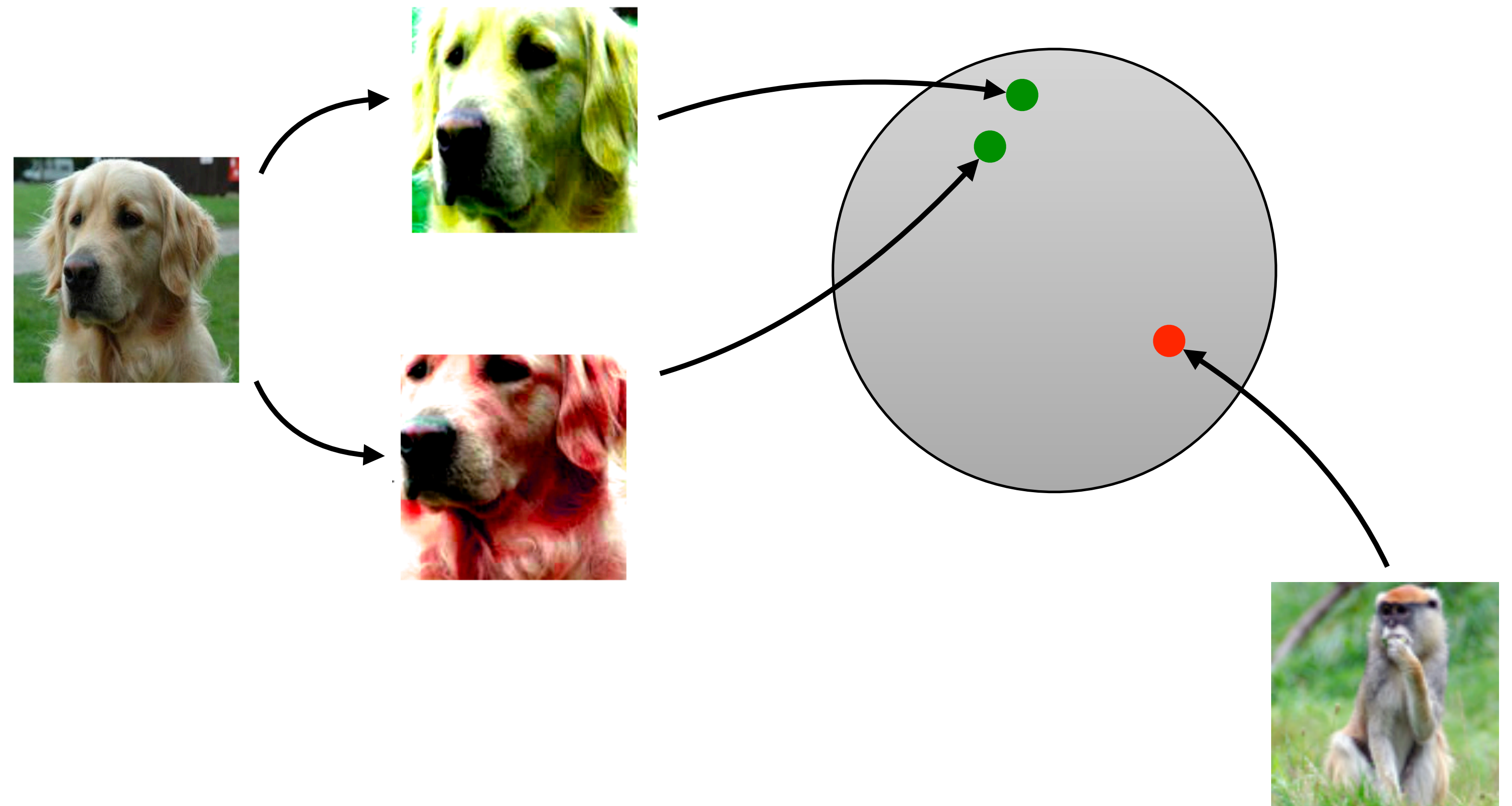
1. Information retrieval (IR), as an application area of representation learning
2. Contrastive learning
3. Approximate nearest-neighbor (ANN) search
4. Neural IR Paradigms: Cross-Attention, Dual-Encoders, Late Interaction

# Why learn representations?

- To improve generalization
- To do more learning from limited high-quality data (transfer learning)
- To **organize information** by exploiting geometric similarity

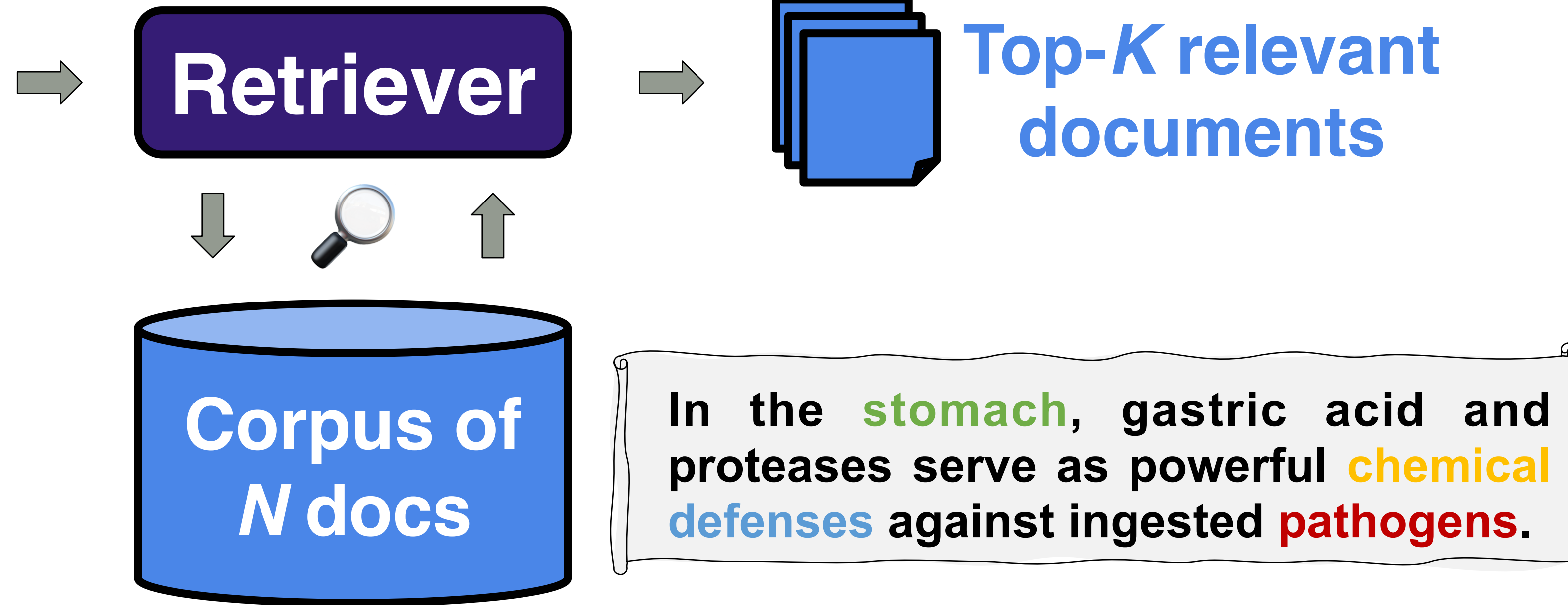
**Q: What inductive biases can we encode in the model architecture and in training to organize information effectively?**

# Information Retrieval



# Information Retrieval at a glance

Query: “What compounds protect the digestive system?”



⚡ **Answer challenging queries accurately**

⚡ **Search billions of docs in milliseconds**

# Retrievers need to have high **quality**

For a given query and the retrieved set of K **results**, we can define some ranking evaluation metrics:

1.  $\text{Success@K} = 1$  if at least one relevant result is in top-K; else 0
2.  $\text{Precision@K} = \# \text{ of relevant items in top K} / K$
3.  $\text{Recall@K} = \# \text{ of relevant items in top K} / \text{Total \# of known relevant items}$

Usually you average these scores across a set of queries!

There are also “graded” relevance metrics like nDCG@K, which are useful when (i) documents have different degrees of relevance and (ii) you care about how documents are ordered within the top-K list.

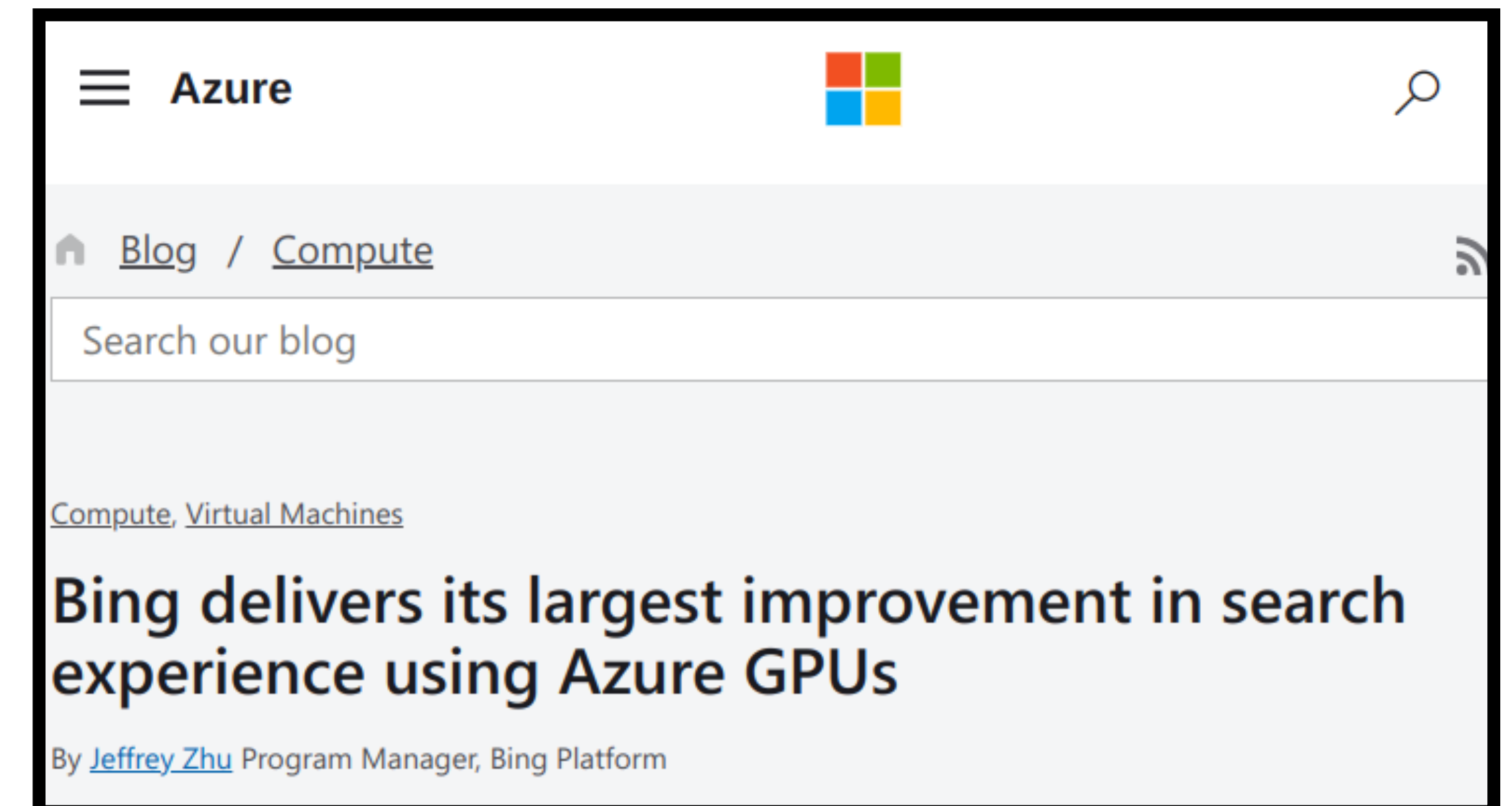
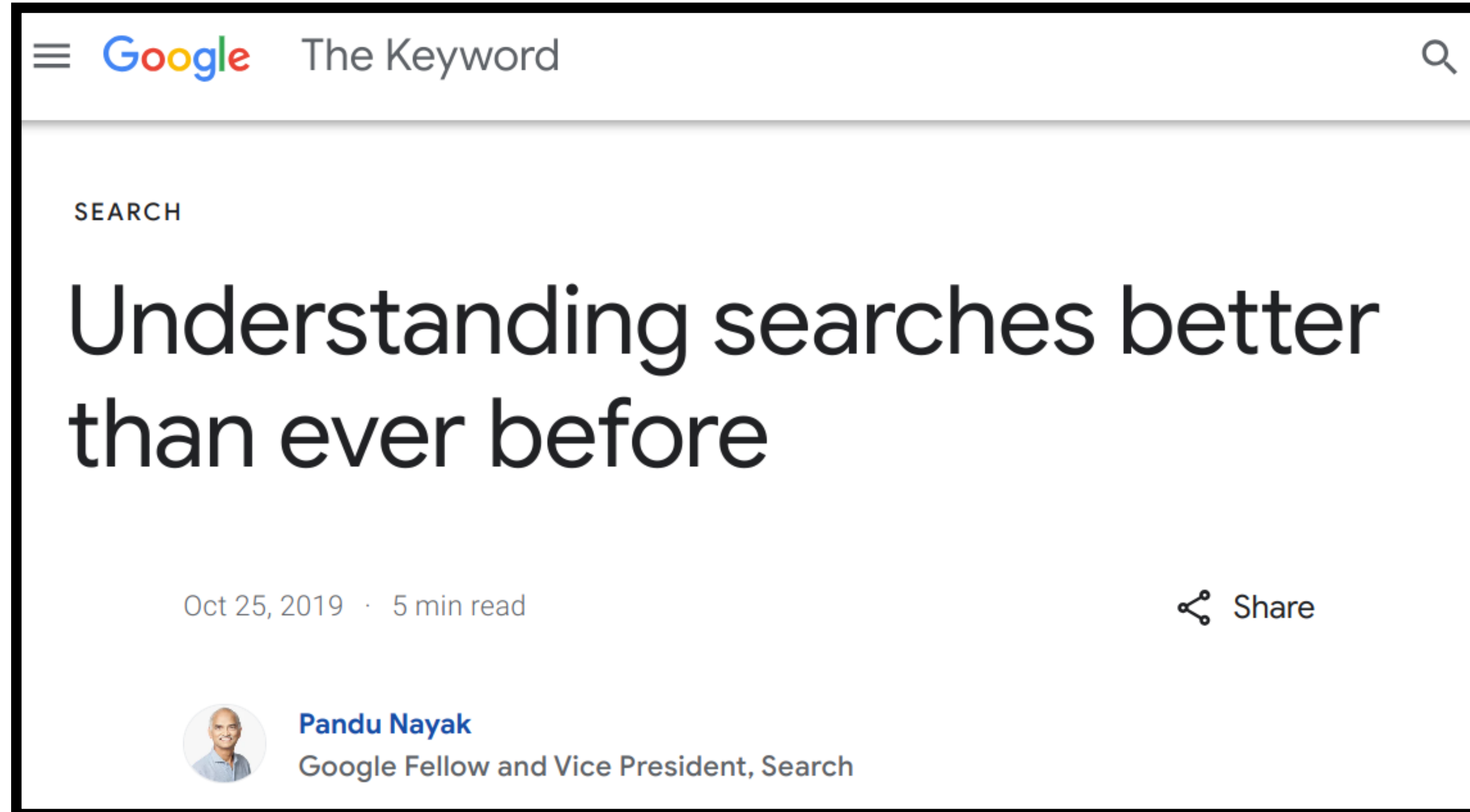
# Retrievers need to have low **latency**

Answering queries within tens or hundreds of milliseconds! In practice, this may also require ***sub-linear*** search time scaling in the corpus size  $N$ .

Today we'll ignore the constants and the cost of “indexing” the corpus (which *are* very important!) and we'll think about things like:

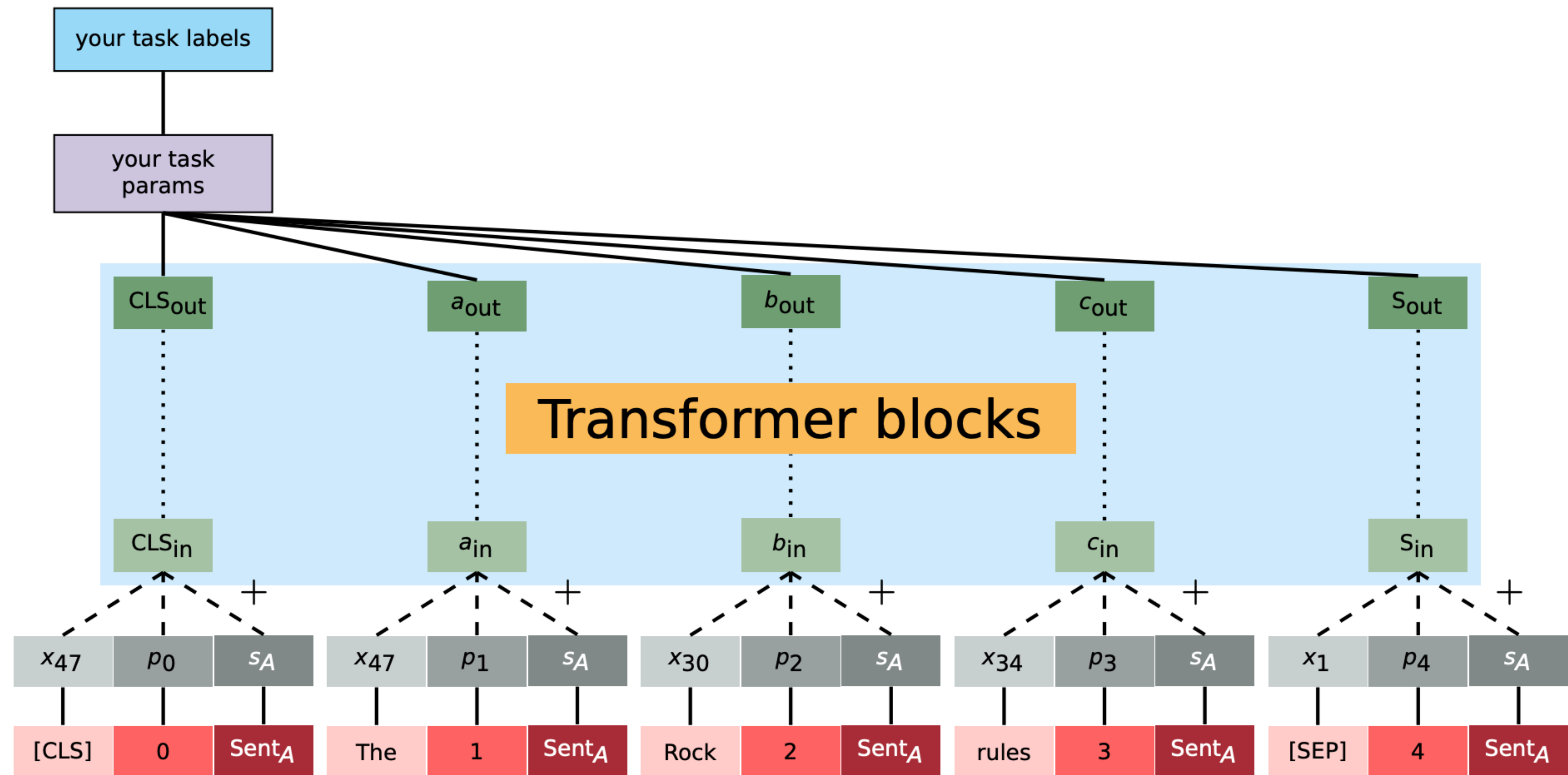
- **$O(N)$**  vs.  **$O(1)$**  *Transformer calls per query*
- **$O(N^2)$**  vs.  **$O(N)$**  vs.  **$O(\sqrt{N})$**  *dot products per query*

# In 2019, pretrained Transformers began transforming IR



# Review: Transformer Encoders (but for language too!)

BERT, trained via masked language modeling ("fill in the blanks")



token + position + "sentence A or B"

# How would you model IR as a deep learning problem?

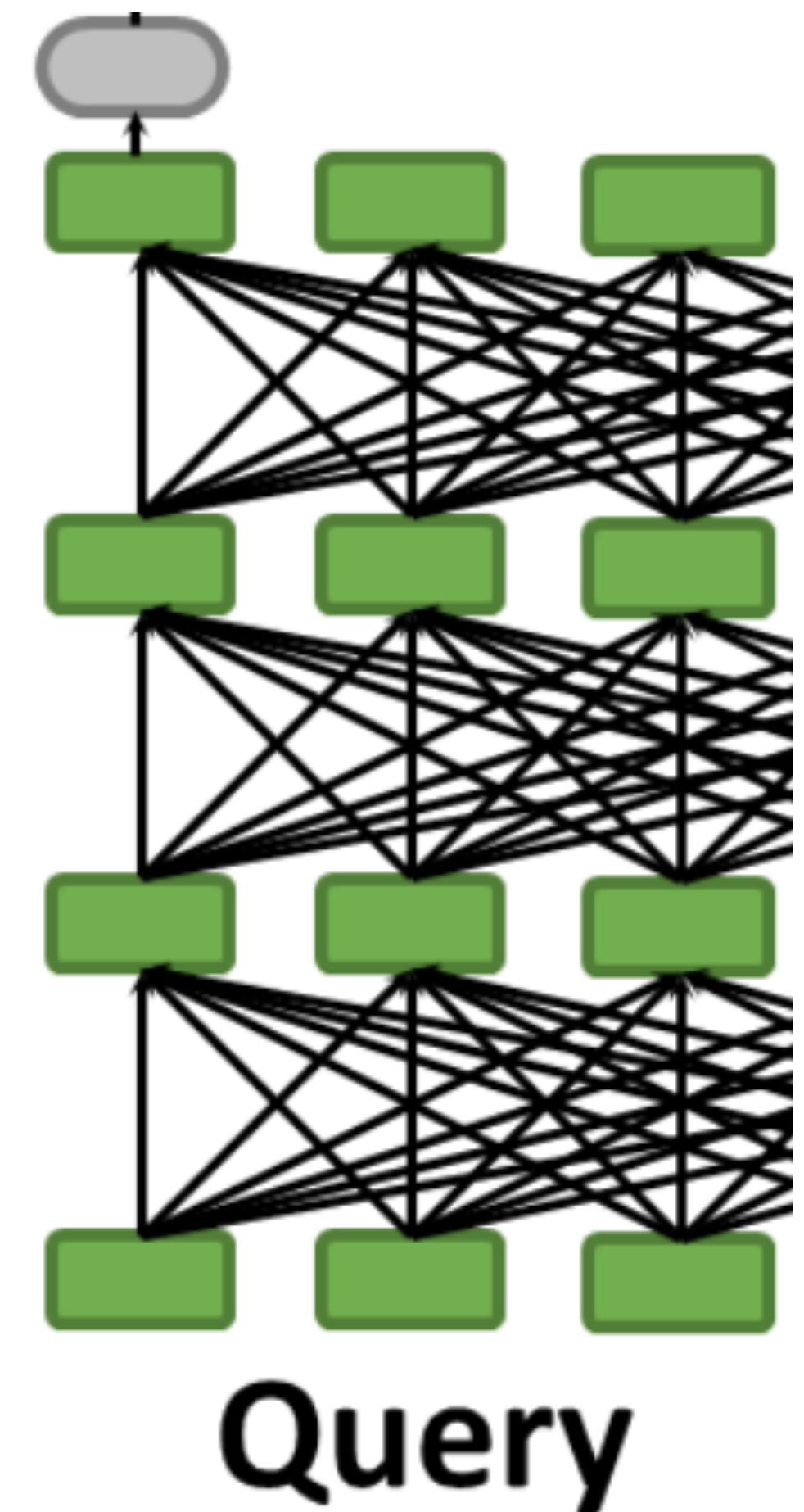
**Idea #1:** What if we do  $N$ -way regression where the input is our query?

**First, what architecture would we use here?**

- RNN or Transformer.

**What's a problem that immediately faces us?**

- Problem: This doesn't compose at all! Learning treats documents as having independent features!



# Can we create a more compositional approach?

**Idea #2:** What if we feed the query and *\*all\**  $N$  documents to the Transformer and map each document's last token to a score?

- Pro: Now the model can compare the query with each document!
- Problem:  **$O(N^2)$  attention** cost will kill us immediately for any sizable  $N$ .

# Can we make a more practical approach?

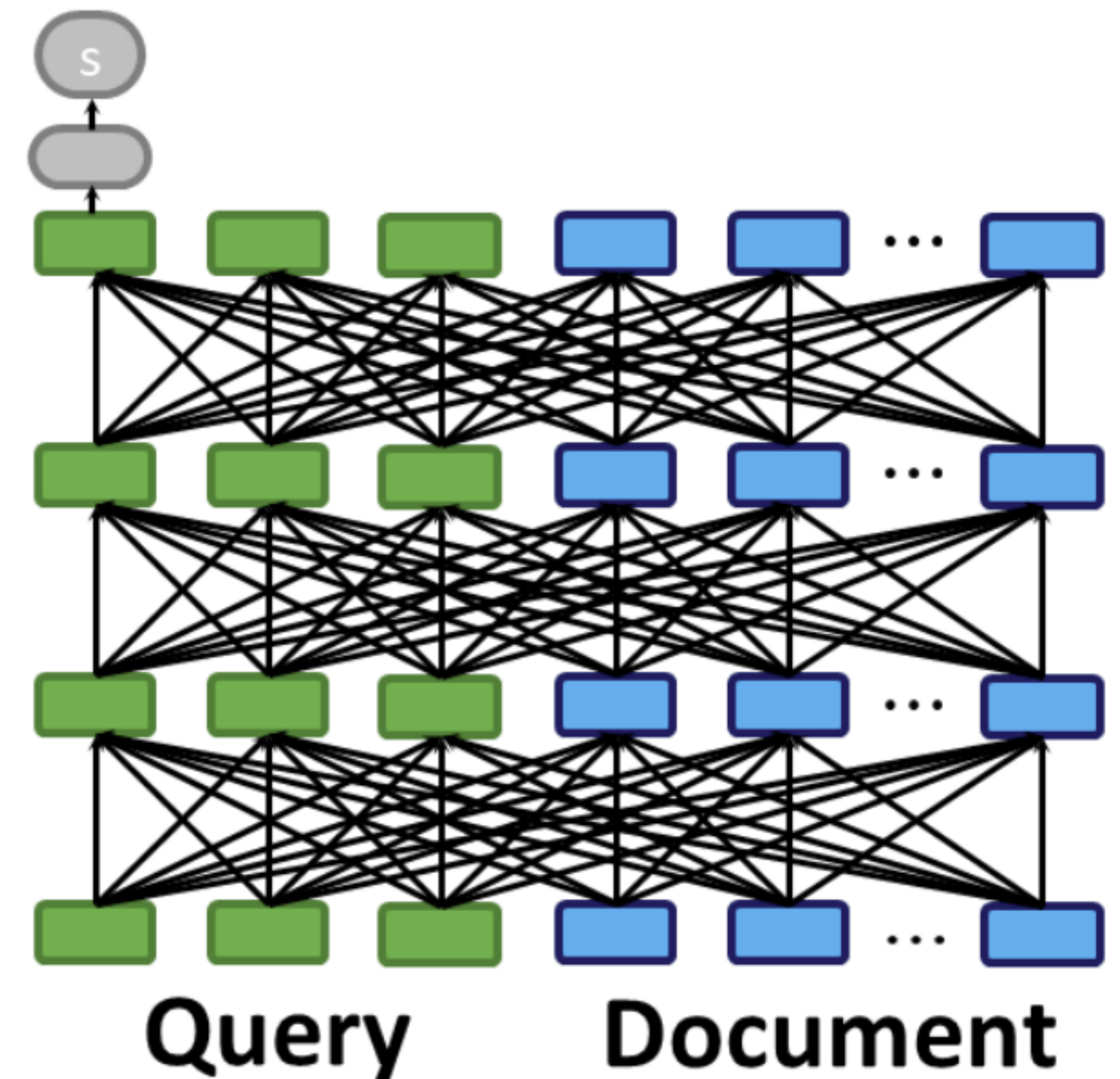
**Idea #3: Decomposition.** This is a trick we'll apply a lot today!

Let's model "pointwise" scoring of each document separately, given the query.

Inductive bias: **doc scores are easy to compute accurately in an independent way.**

Is this true? Not necessarily...? But without something like it, modeling may be infeasible!

(or is it?)



$$S = \text{Transformer}(Q; D)$$

# Cross-Encoders: Pointwise Attention

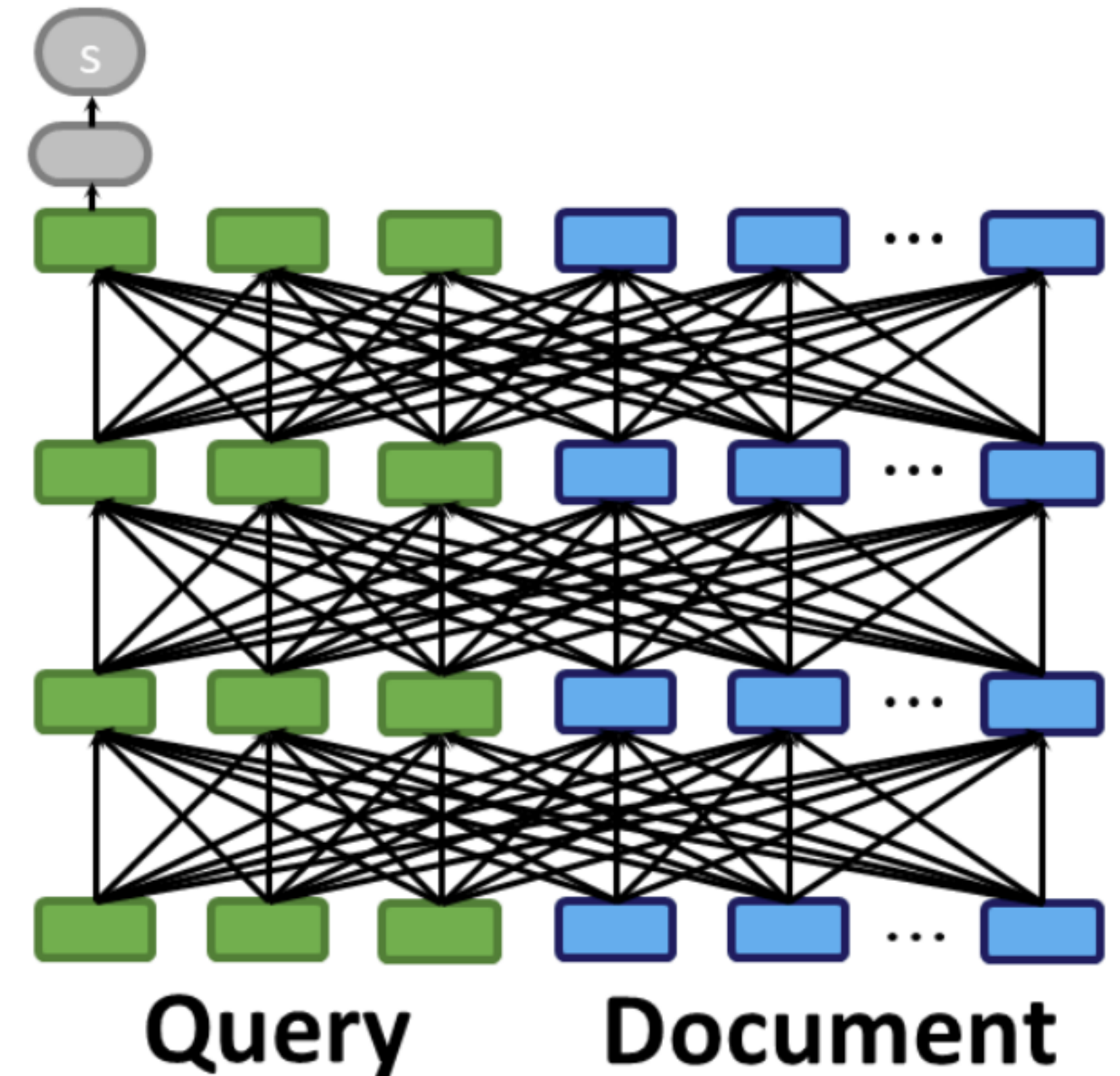
## How do we get a training set for this?

1. Ask humans to label all  $N$  documents per query?
2. Ask humans to label 1-2 “relevant” documents for each query. Assume ~everything else is irrelevant.

## How do we train on these labels?

- A. Train given a query and *ALL* documents?
- B. *Decompose!* Mini-batches per query.

For each  $q$ , sample  $d$ , and train a linear layer atop the Transformer to map output vector to a score.



$$S = \text{Transformer}(Q; D)$$

# Problems with this cross-encoder design

For one, it's extremely expensive:  **$O(N)$  Transformer calls** for EACH QUERY.

Can it be useful? Yes. If we can get the top- $\{10 \cdot K\}$  with a fast method, we can re-rank them to get the "best" top- $K$  in there!

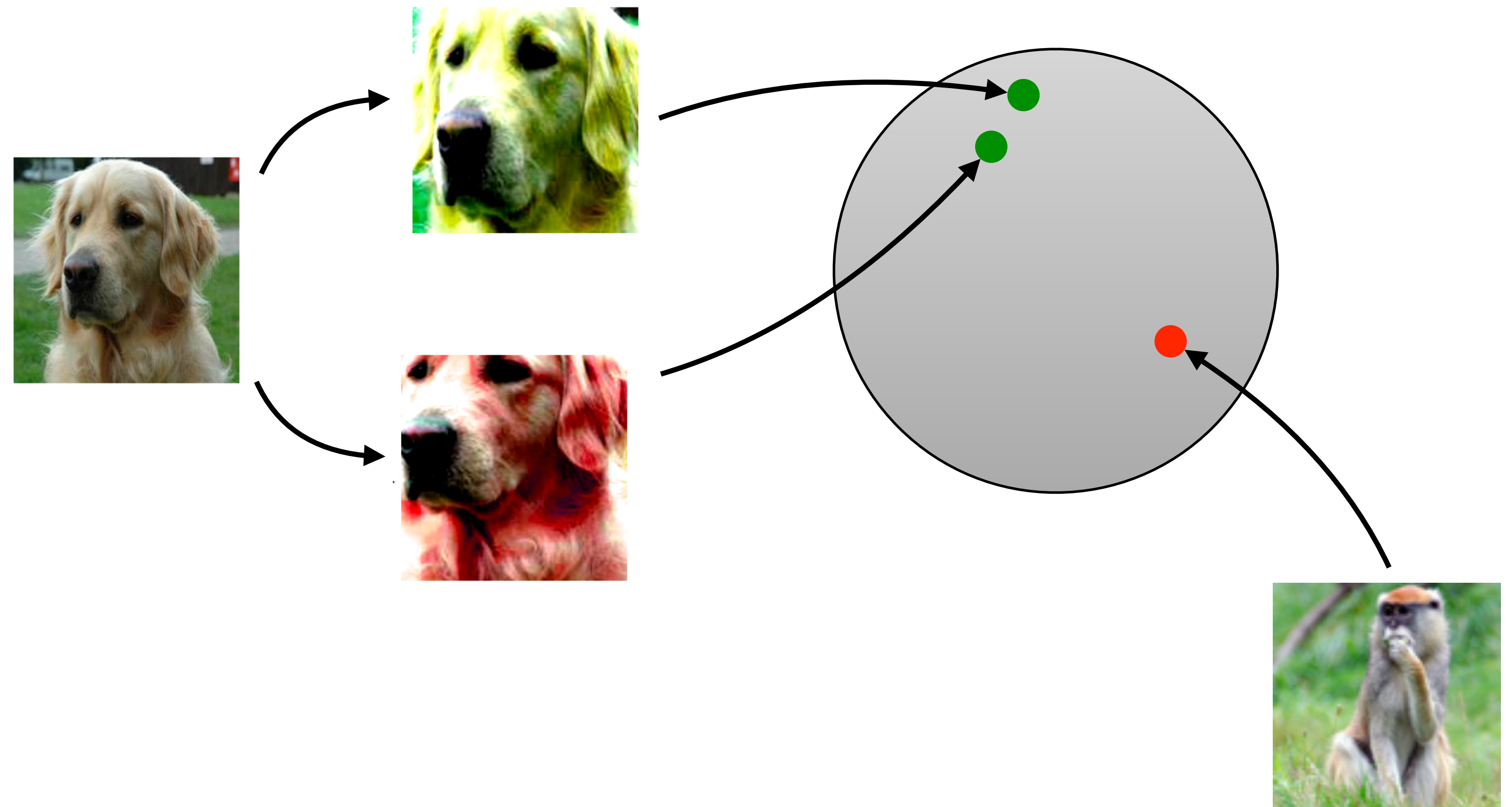
**Can we get *much* better latency?**

*Decompose again!* We already know all the documents ahead of time.

What if we encode each  $d$  once and for all, independent of the query?

*Inductive Assumption: The document representation doesn't depend on the query! Is this true?*

# Dual Encoders & Contrastive Learning



# Vector Similarity (aka Dual-Encoders)

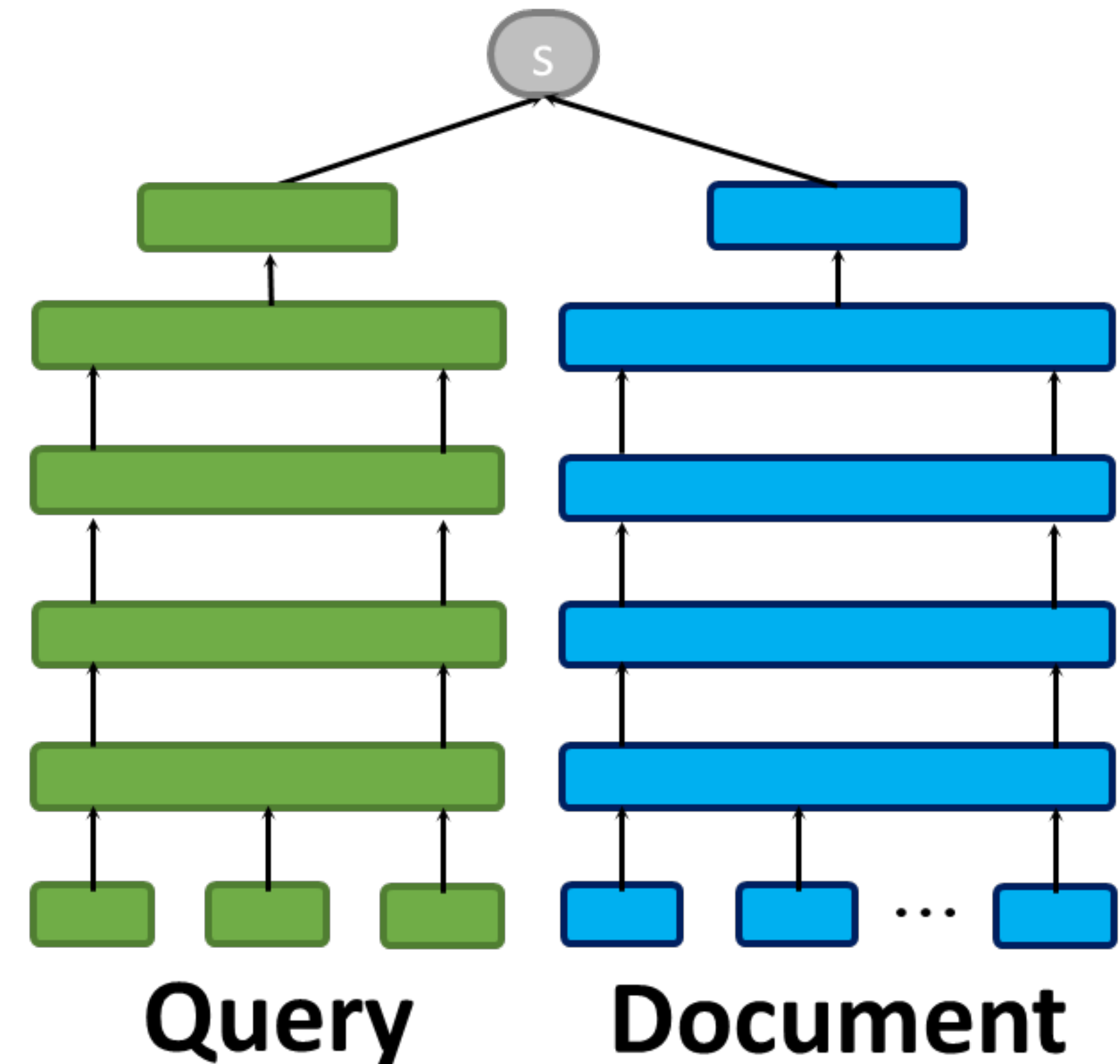
$$\text{sim}(q, p) = E_Q(q)^\top E_P(p)$$



We can pre-compute document representations!

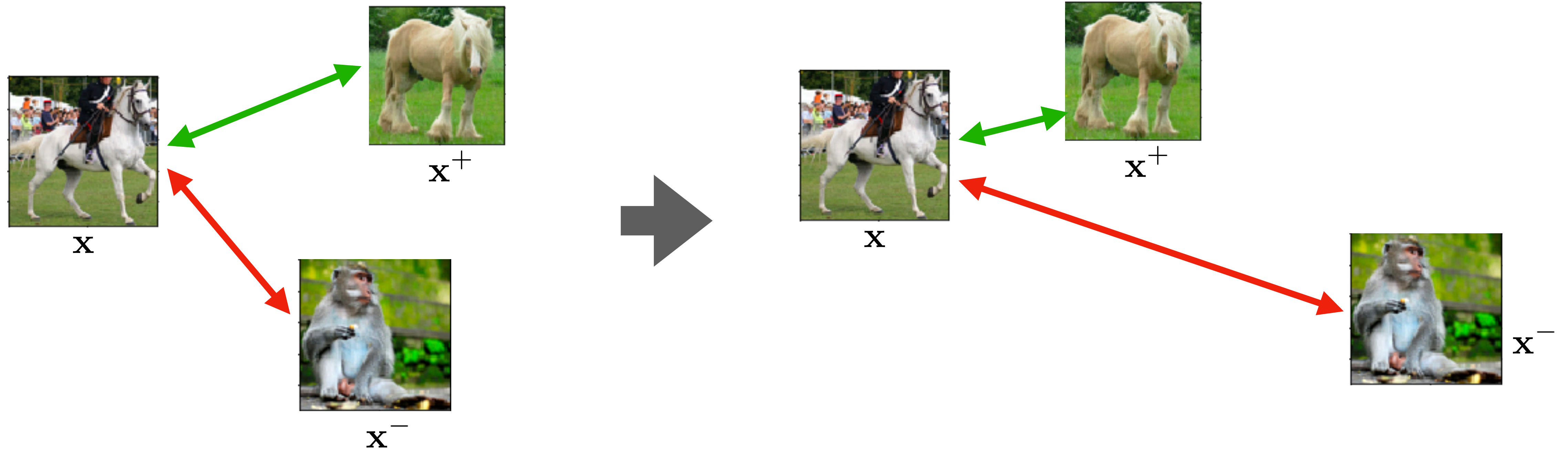
At query time, we need only ONE Transformer forward pass, i.e. to encode the query itself.

**How do we train this? Pointwise regression is not necessarily good or practical. (Why not?)**

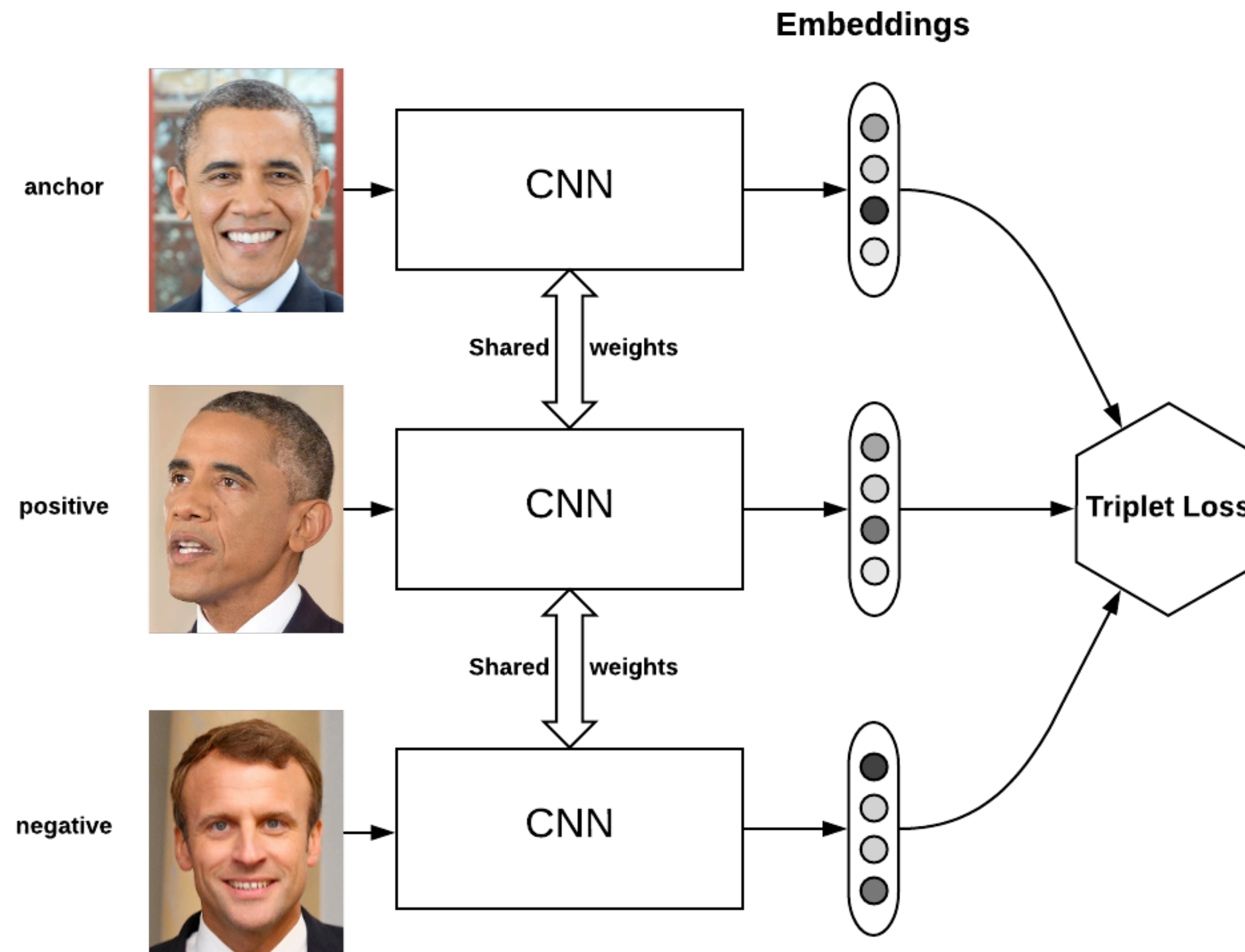


# Contrastive losses

**distance of dissimilar pair(s)  $\gg$  distance of similar pair(s)**



# Factorized view for image representations

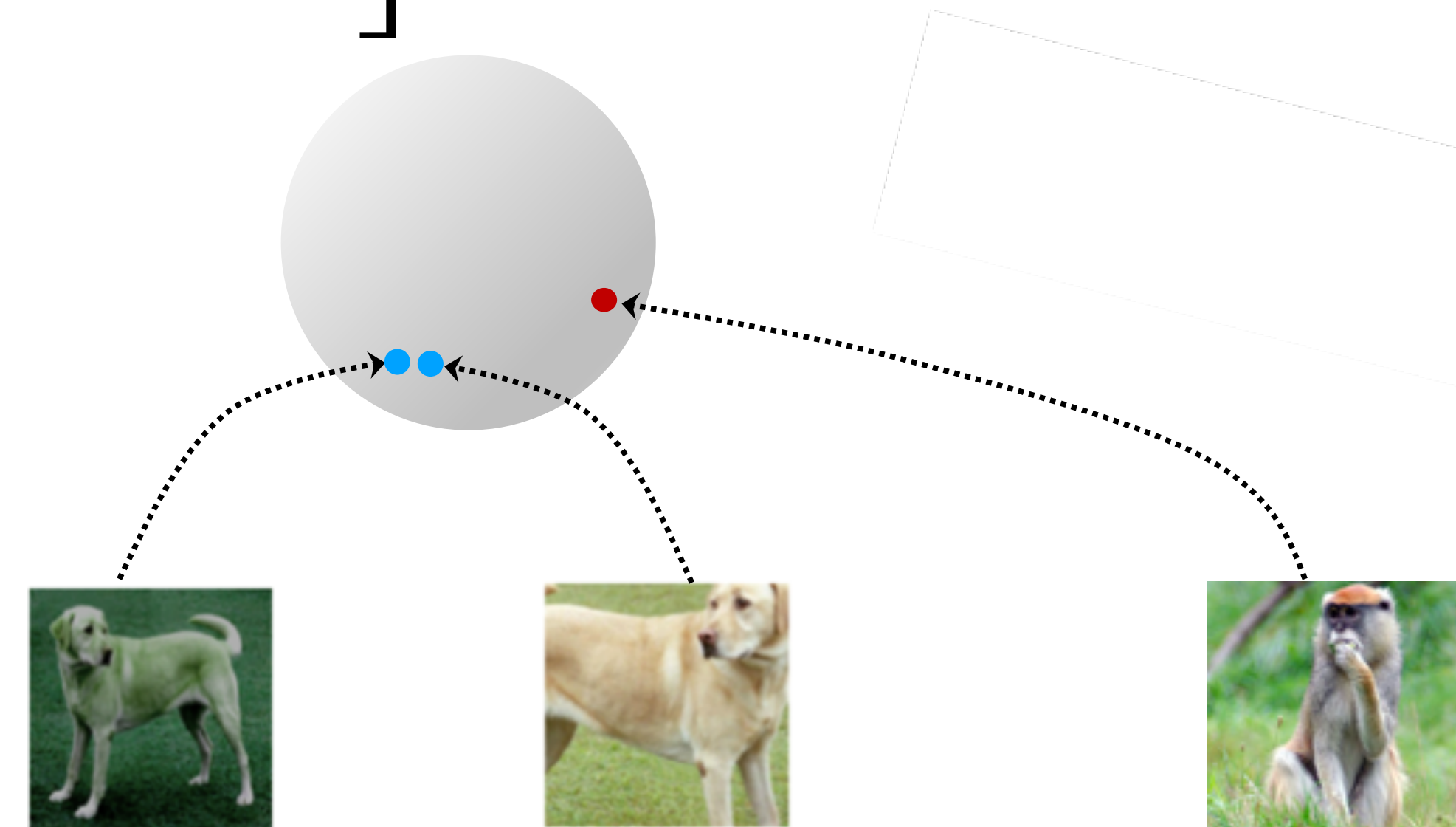


# Common setup: InfoNCE loss

- Encoder maps data onto a hypersphere:  $f: \mathcal{X} \rightarrow \mathbb{S}^{d-1}$
- Cross-entropy loss with softmax as a “classifier” for relevance

$$\min_f \mathbb{E}_{(\mathbf{x}, \mathbf{x}^+) \sim p_{pos}, \{\mathbf{x}_i^-\}_{i=1}^N \sim p_{data}} \left[ -\log \frac{e^{f(\mathbf{x})^\top f(\mathbf{x}^+)/\tau}}{e^{f(\mathbf{x})^\top f(\mathbf{x}^+)/\tau} + \sum_{i=1}^N e^{f(\mathbf{x})^\top f(\mathbf{x}_i^-)/\tau}} \right]$$

*Note “N” is overloaded here (# negatives << corpus size)  
[van den Oord et al 2018]*



# Common setup: InfoNCE loss

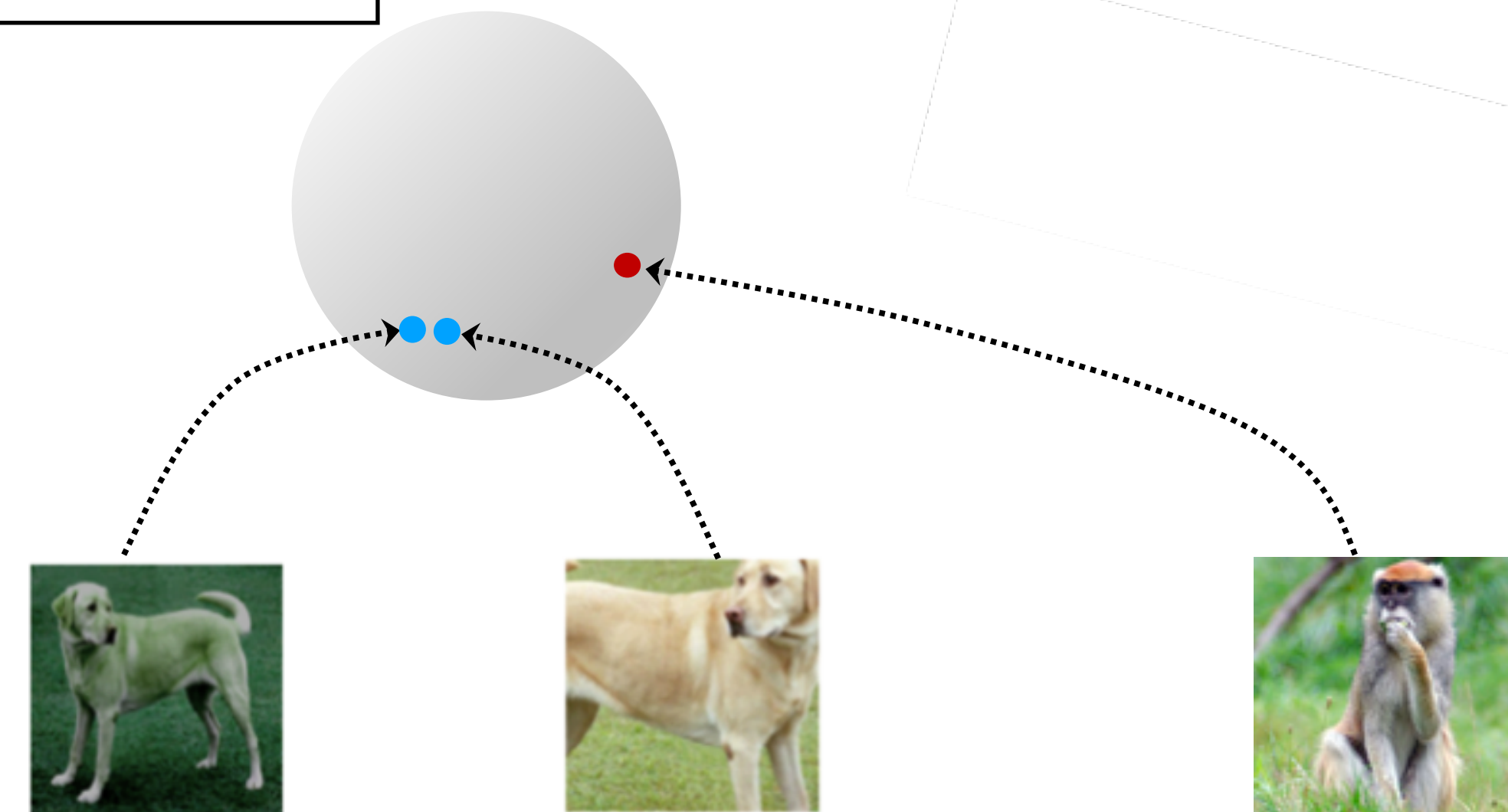
- Encoder maps data onto a hypersphere:  $f: \mathcal{X} \rightarrow \mathbb{S}^{d-1}$
- Cross-entropy loss with softmax as a “classifier” for relevance

$$\min_f \mathbb{E}_{(\mathbf{x}, \mathbf{x}^+) \sim p_{pos}, \{\mathbf{x}_i^-\}_{i=1}^N \sim p_{data}} \left[ -\log \frac{e^{f(\mathbf{x})^\top f(\mathbf{x}^+)/\tau}}{\underbrace{e^{f(\mathbf{x})^\top f(\mathbf{x}^+)/\tau}}_{\text{similarities}} + \sum_{i=1}^N e^{f(\mathbf{x})^\top f(\mathbf{x}_i^-)/\tau}} \right]$$

cross-entropy loss

softmax

Note “N” is overloaded here (# negatives << corpus size)  
[van den Oord et al 2018]



# Common setup: InfoNCE loss

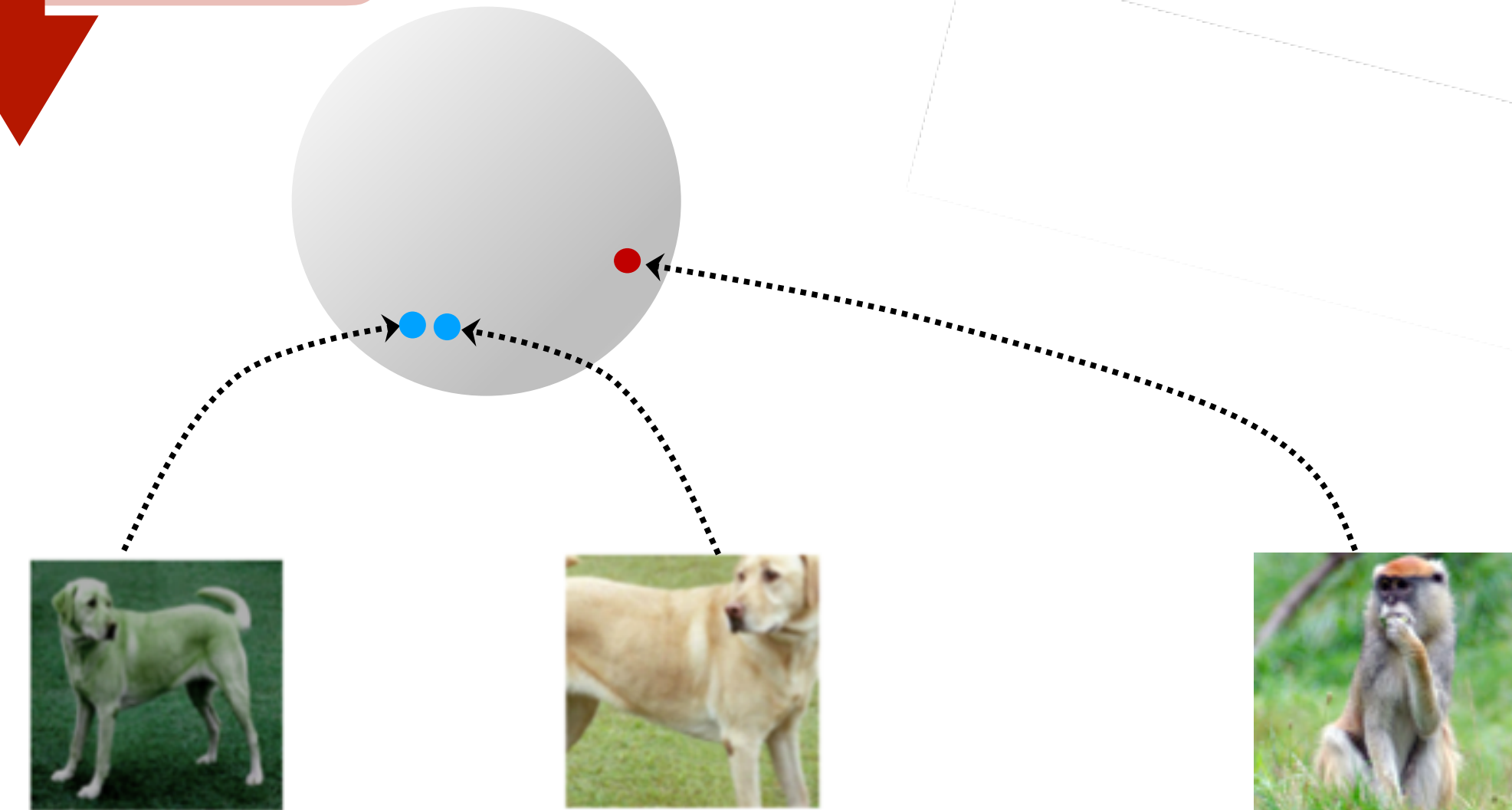
- Encoder maps data onto a hypersphere:  $f: \mathcal{X} \rightarrow \mathbb{S}^{d-1}$
- Cross-entropy loss with softmax as a “classifier” for relevance

$$\min_f \mathbb{E}_{(\mathbf{x}, \mathbf{x}^+) \sim p_{pos}, \{\mathbf{x}_i^-\}_{i=1}^N \sim p_{data}} \left[ -\log \frac{e^{f(\mathbf{x})^\top f(\mathbf{x}^+)/\tau}}{e^{f(\mathbf{x})^\top f(\mathbf{x}^+)/\tau} + \sum_{i=1}^N e^{f(\mathbf{x})^\top f(\mathbf{x}_i^-)/\tau}} \right]$$

*pull positive pair together*

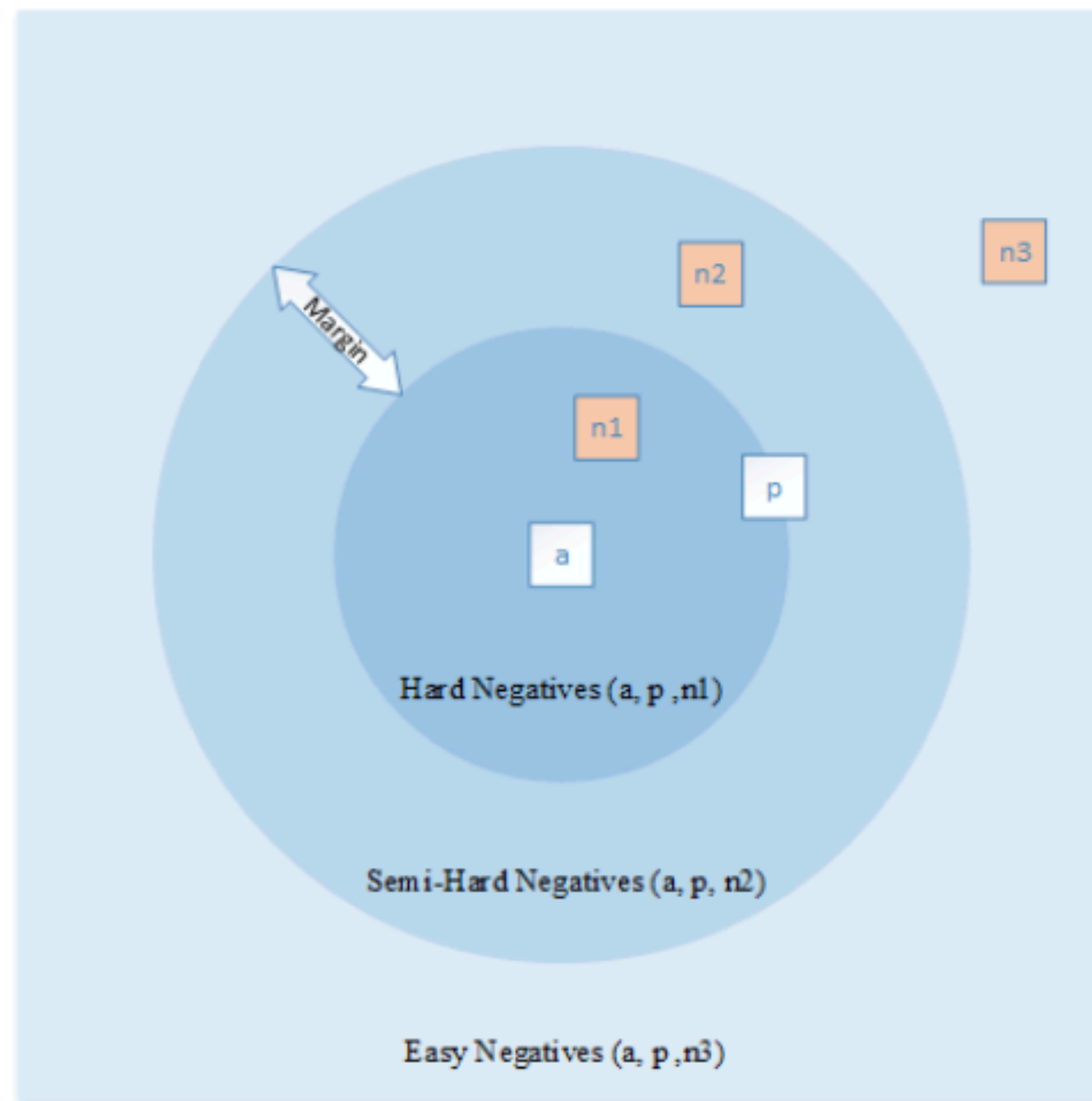
*push negative pairs apart*

Note “N” is overloaded here (# negatives  $\ll$  corpus size)  
[van den Oord et al 2018]



# Should we sample the negative documents randomly?

- “Hard” negatives instead of random ones.
- Currently misplaced, i.e. closer to anchor than a positive example!



Hard Negative Mining

$$d(a, n) < d(a, p)$$

Semi-Hard Negative Mining

$$d(a, p) < d(a, n) < d(a, p) + \text{margin}$$

Easy Negative Mining

$$d(a, p) + \text{margin} < d(a, n)$$

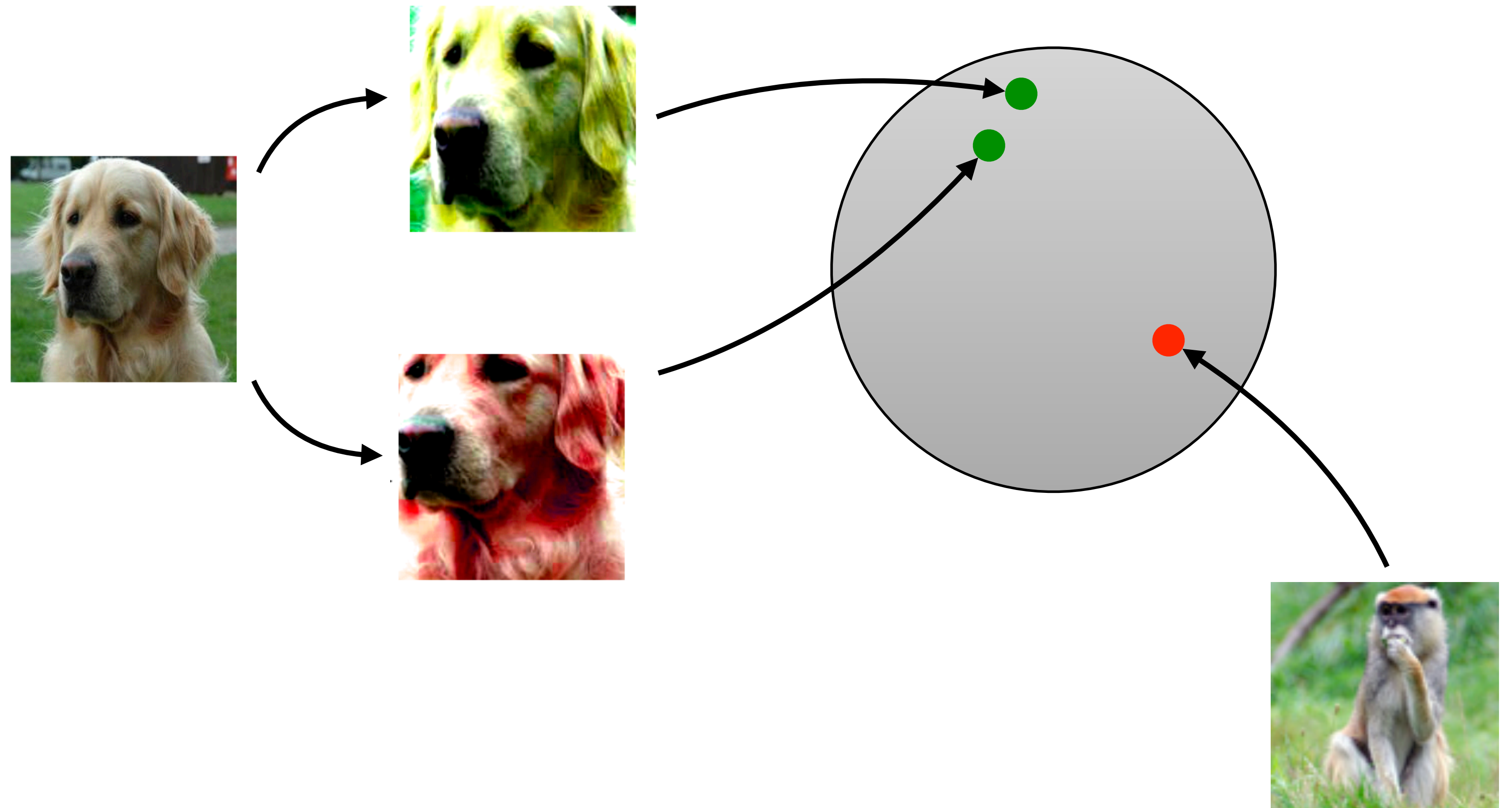
**Figure 4.** Negative Mining.

# Are (even) Dual-Encoders fast enough?

So far, using a Dual-Encoder still involves  **$O(N)$  dot products** for each query, though it's only  **$O(1)$  Transformer calls** at query time.

Can we do (much) faster?

# Approximate Nearest-Neighbor (ANN) Search



# A simple heuristic approach: Inverted File ("IVF")\*

## At indexing time:

1. Cluster all  $N$  document vectors into  $\sqrt{N}$  groups (e.g., via k-means).
2. Store an "inverted list" per cluster that enumerates all the documents in that cluster.  
*We'll assume here that we can get roughly equal-sized clusters (size  $\sqrt{N}$  each)!*

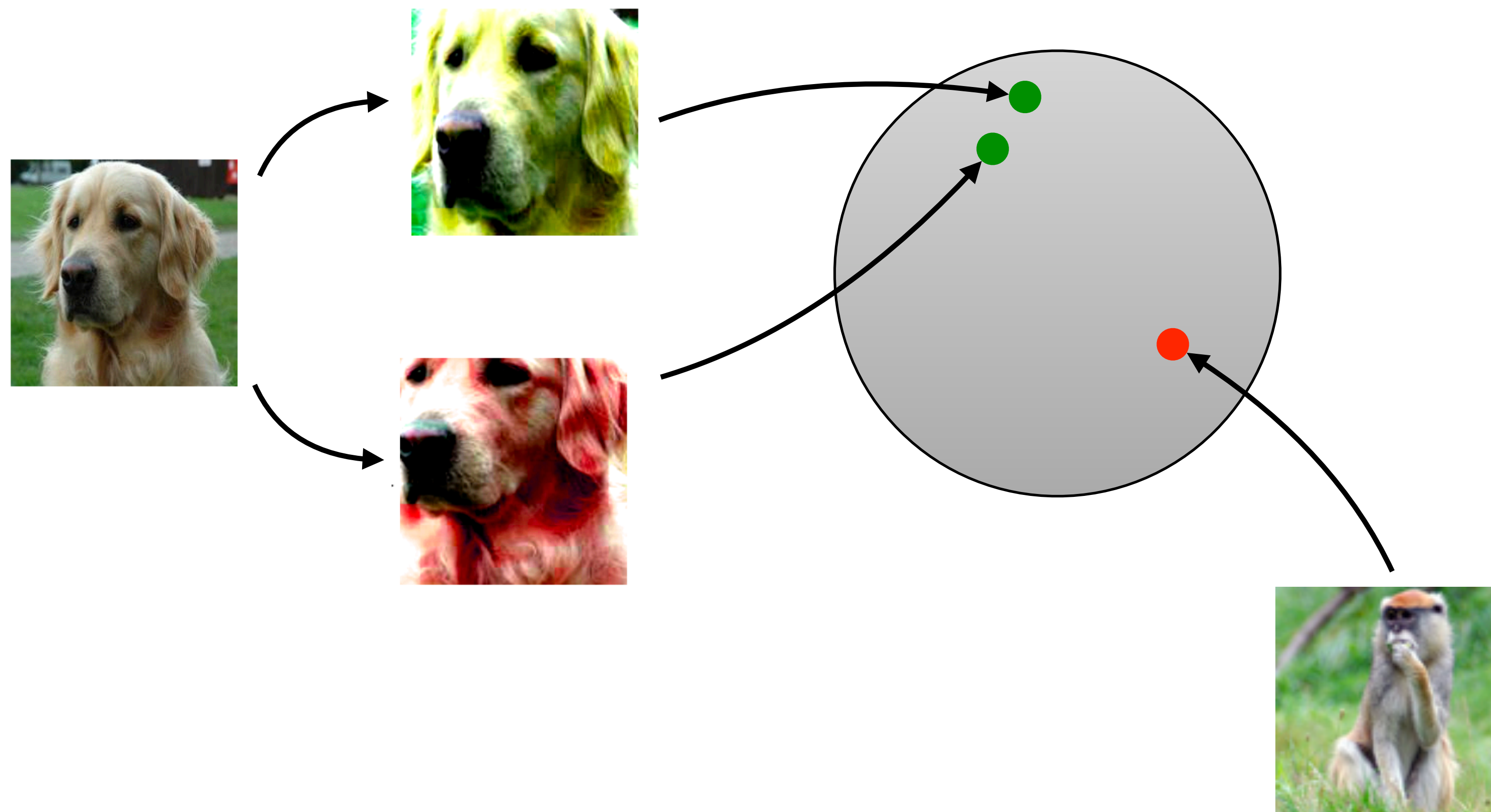
## At query time:

*OK, so it's really fast. But is it high-quality enough?*

1. Compute distance from the query to all  $\sqrt{N}$  centroids. Pick the closest centroid.
2. Only scan that list and compute dot products for the  $O(\sqrt{N})$  documents in there.

\* [Bonus] Other approaches use more space but have better approximation: HNSW, LSH, ...

# Limitations of a (single) dense vector representation



# [Bonus] How expressive is a Dual-Encoder?

1. **Dual-encoder universality.** A dot-product dual-encoder Transformer can approximate any continuous scoring function *if the embedding dimension and the encoder are large enough*. [Menon et al. 2022]
2. **Dimension lower bound.** For any dim  $d$ , it is easy to design IR tasks (specifically, query-document relevance mappings) where  $d$ -dimensional dual-encoders fails empirically and theoretically\*! [Weller et al. 2025]

\*The authors of “Global Minimizers of Sigmoid Contrastive Loss” (BBNP 2025), another recent relevant paper that studies the necessary dimension for representations for image-caption settings, remark to me that this cannot hold if we only care about **top-k retrieval**. Classical combinatorics results show that with only  $2 \cdot k$  dimensions, any number  $N$  of documents can be embedded such that **any subset of  $k$**  of them can be linearly separated from the rest.

That said, the sign-rank result of Weller et al. establishes a lower bound on dimension if you want your dual-encoder to be able to **separate \*all\* relevant vs. irrelevant documents for each query**. (You may imagine cases where either one is suitable.)

**"You can't cram the meaning of a whole %&!\$# sentence into a single \$&!#\* vector!"** (by Ray Mooney)

This was a statement in my opening invited talk at the [ACL 2014 Workshop on Semantic Parsing](#), the slides are available on this site. As I said at the talk, you can use your language model of informal English to fill in the masked portions.

*In Defense of Dual-Encoders: Menon et al. 2022*  
*Theoretical Limitations of Embeddings: Weller et al. 2025*  
*Global Minimizers of Sigmoid Contrastive Loss: BBNP 2025*

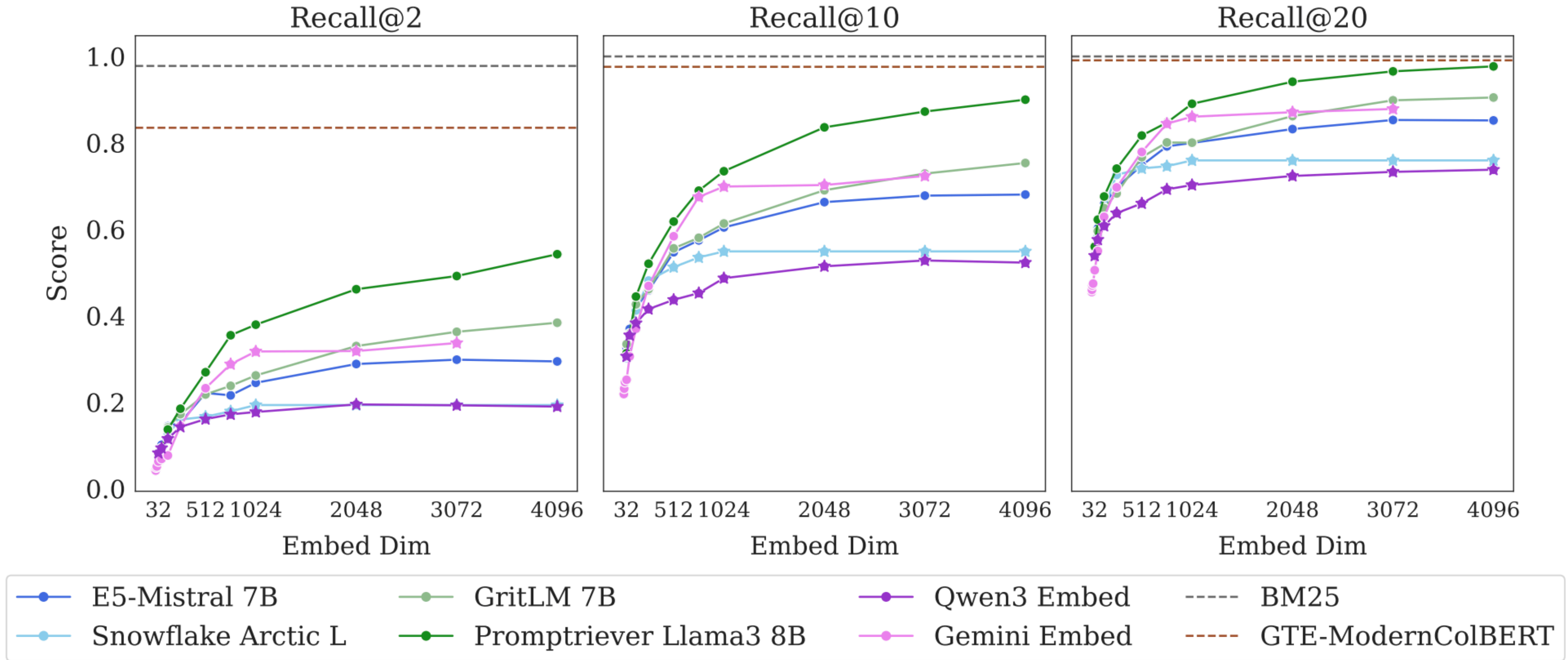
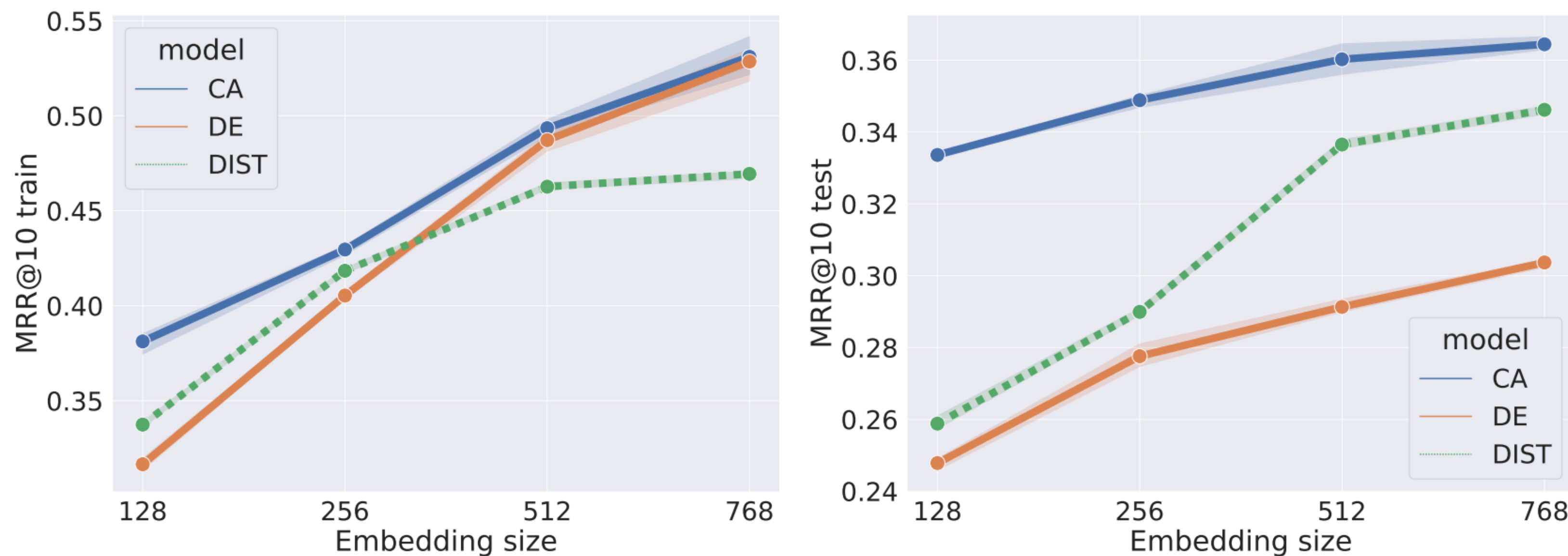


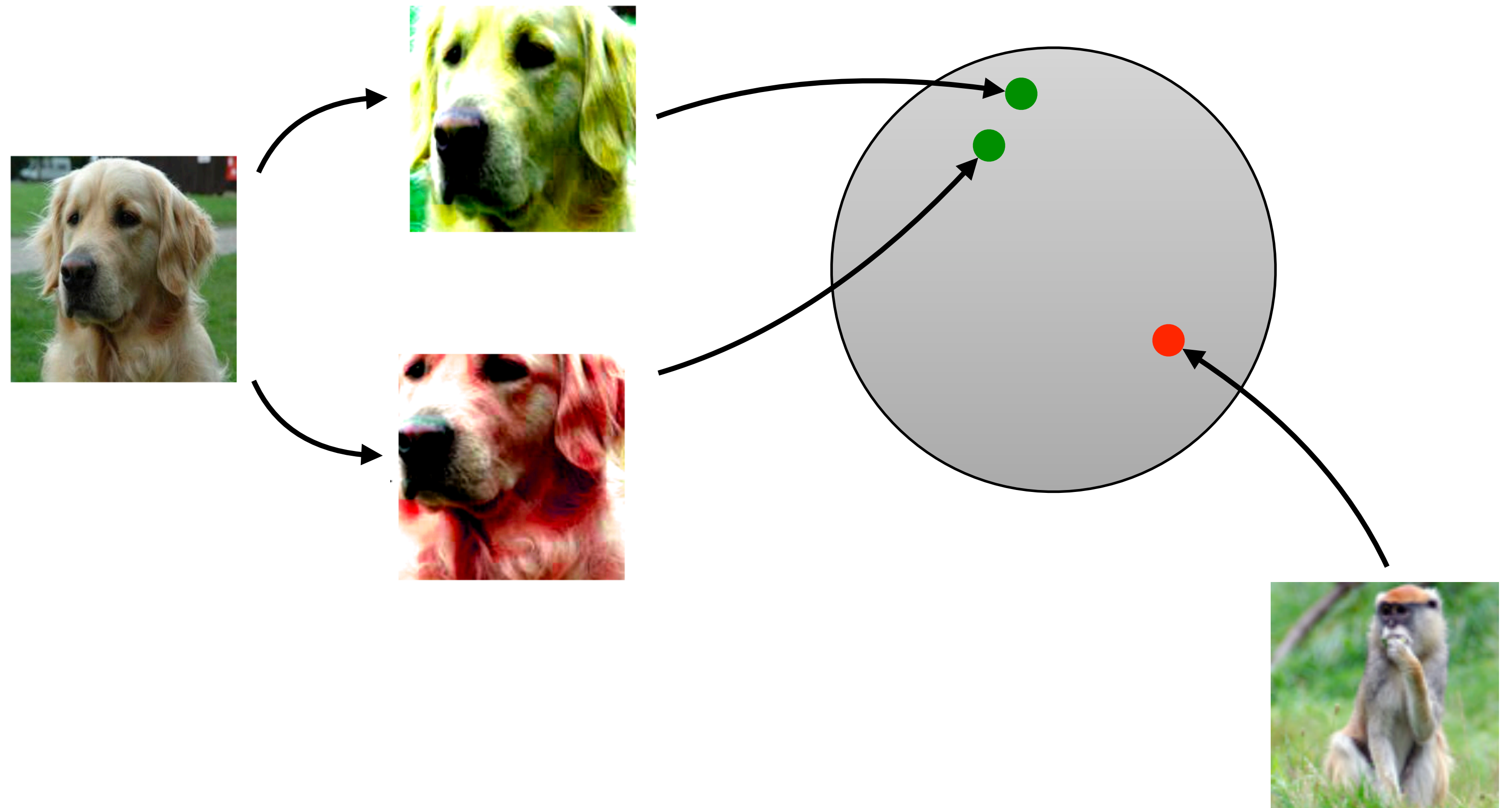
Figure 4 | Scores on the LIMIT small task (N=46) over embedding dimensions. Despite having just 46 documents, model struggle even with recall@10 and cannot solve the task even with recall@20.

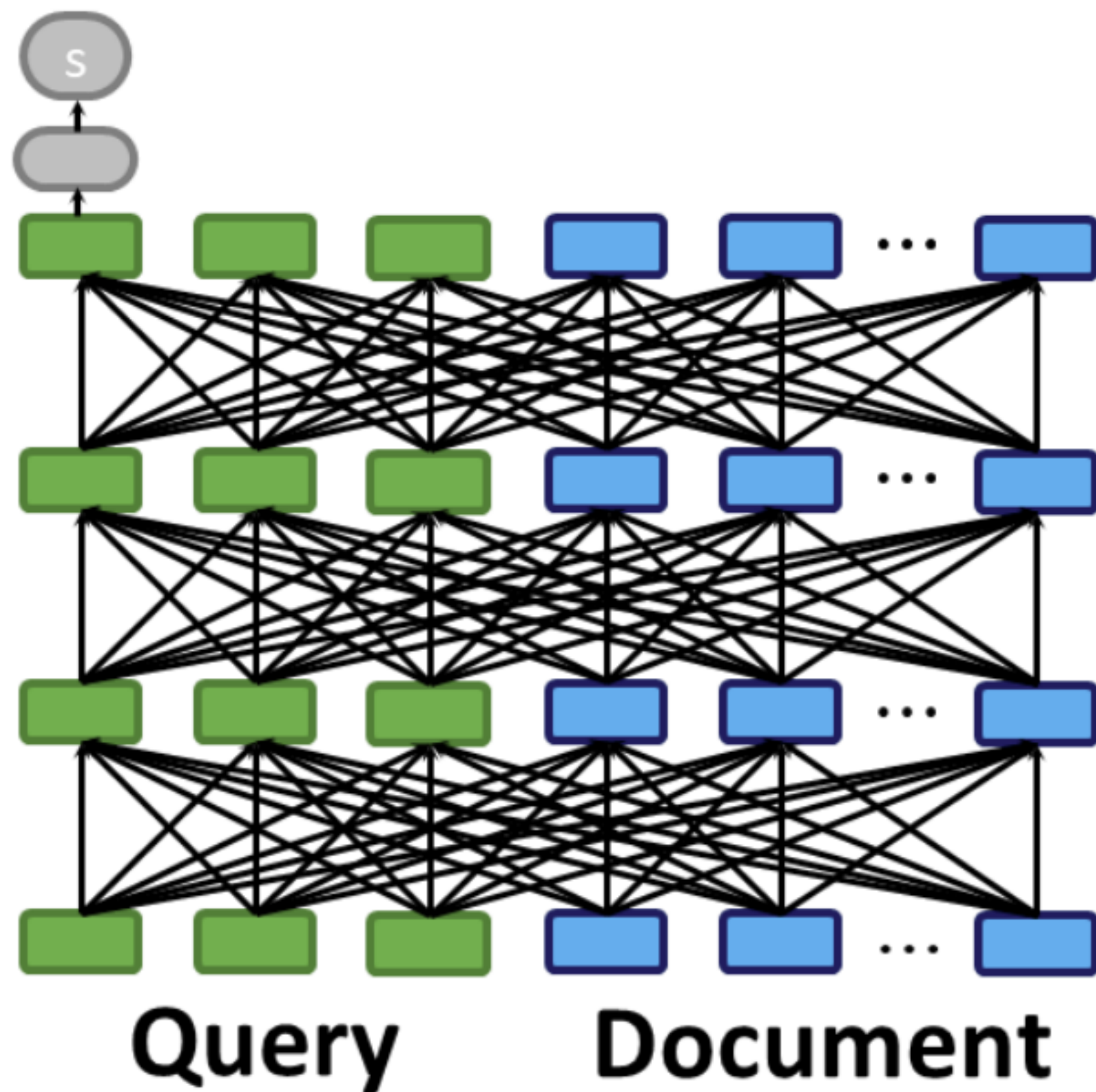
*In practice... for simple tasks, it's often a **generalization** issue.*

1. We often see low error on the training set — but poor generalization!
2. Intuition: Blunt representation leads to blunt updates. The query vector is shared across *all* documents. To lower the score for a negative  $\langle q, d \rangle$  pair, the update moves  $q$  elsewhere. Often, close to *other* negative docs!



# Late Interaction (aka can we get the best of both worlds?)

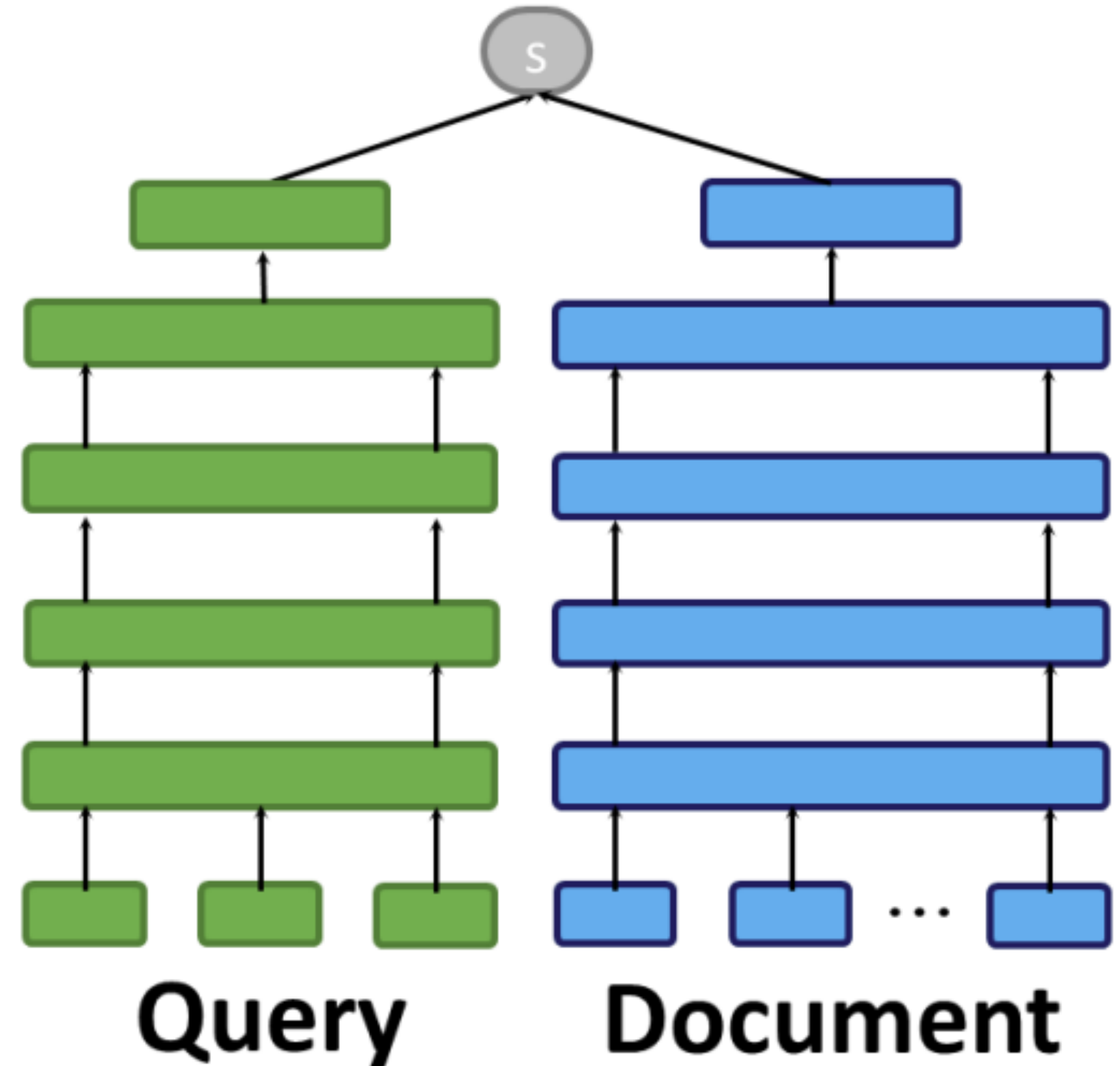




## Cross Attention

✓ High Quality

✗ Unscalable



## Single-Vector Dual-Encoder

✓ Highly Scalable

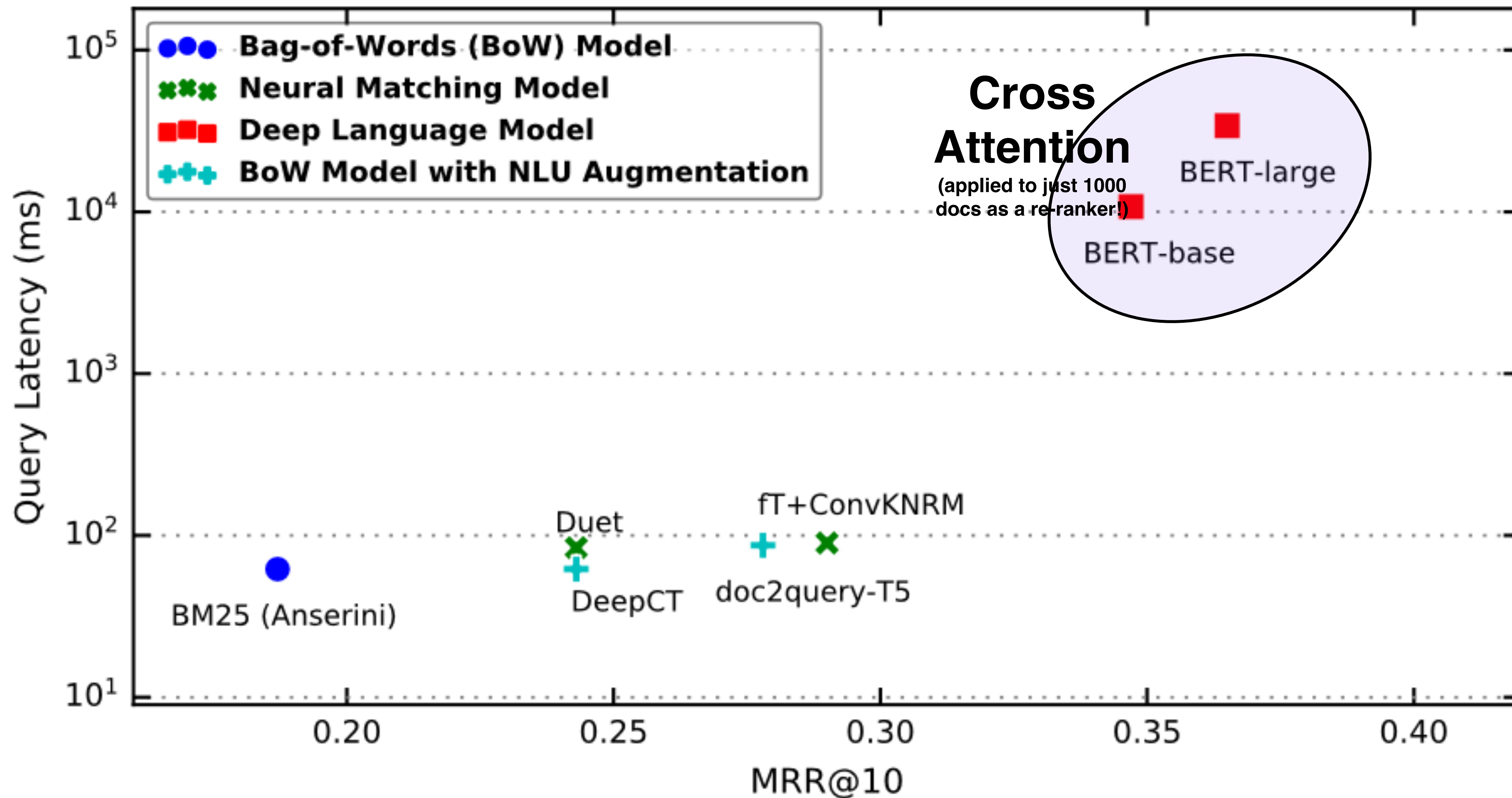
✗ Low Quality

# ColBERT: The Late Interaction Paradigm

- Hypothesis: The main value of attention for IR is the **fine-grained interactions**.
- In particular, jointly encoding queries & documents is redundant for IR tasks.
- But modeling similarity as a dot product is a major bottleneck for learning!

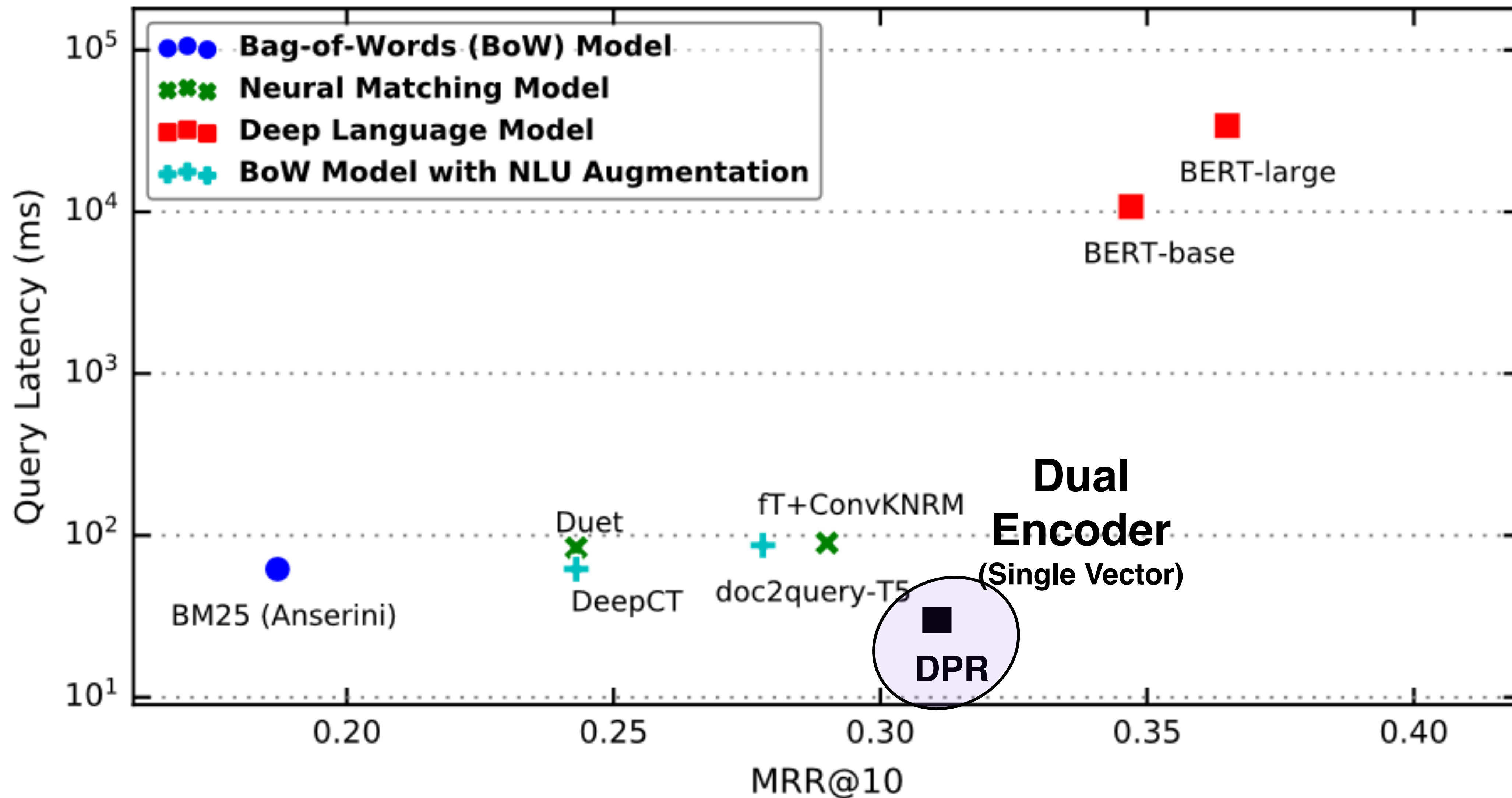
Let's get a sense of the tradeoffs here.

**Lower Latency**  
*(log-scale)*



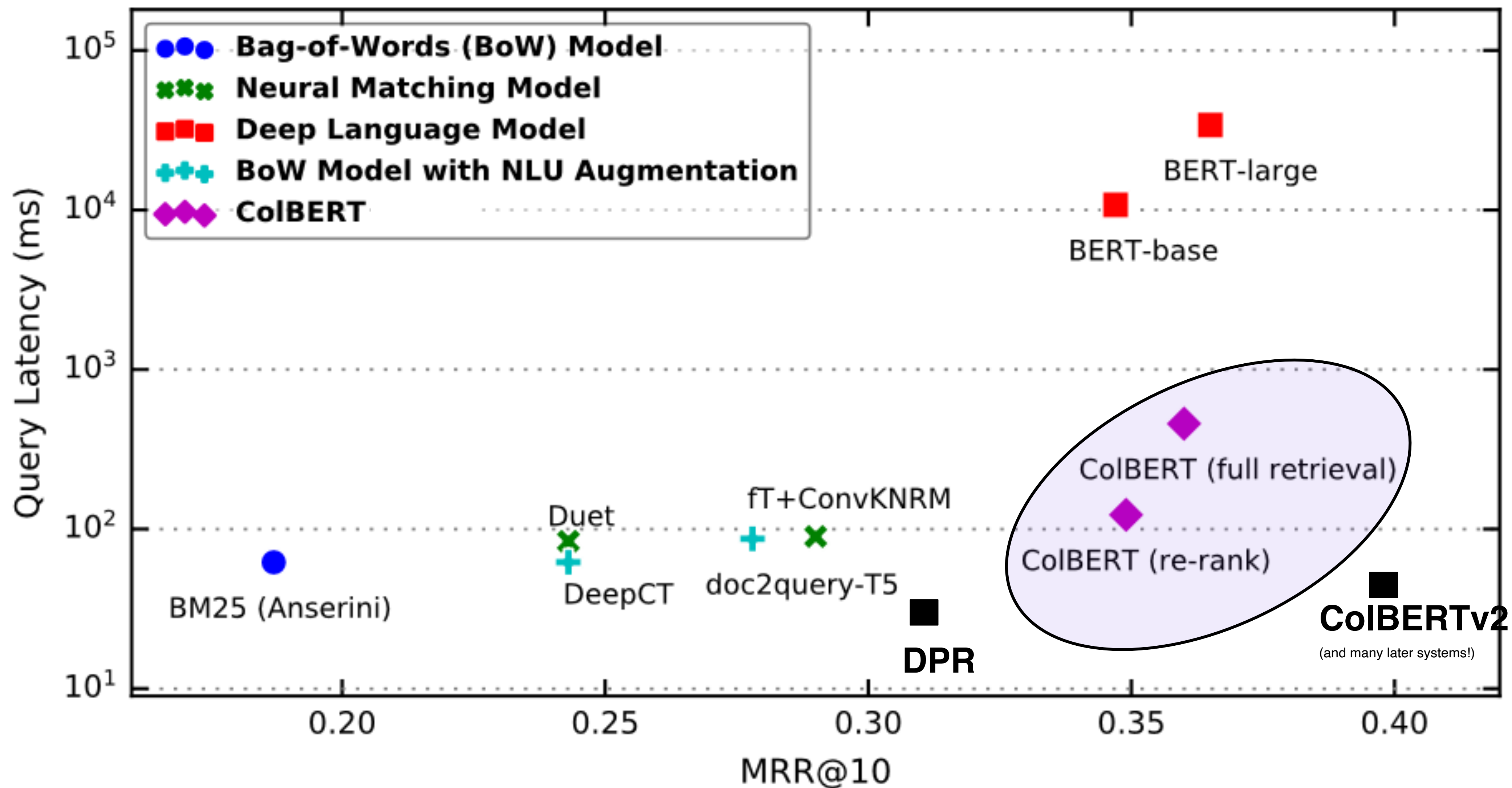
**Higher quality**

**Lower Latency**  
*(log-scale)*



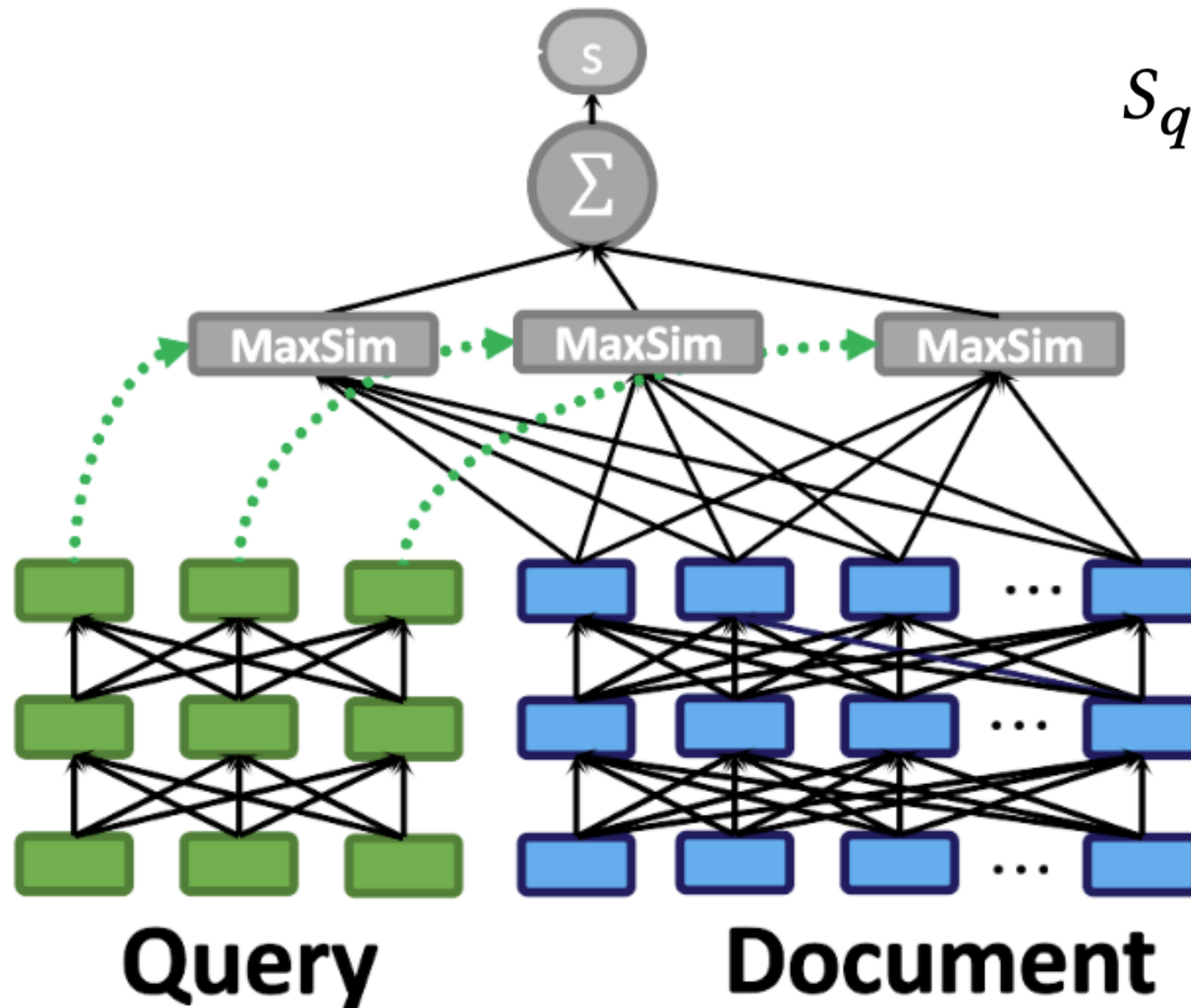
**Higher quality**

Lower Latency  
(log-scale)



Higher quality

# ColBERT: The Late Interaction Paradigm



$$S_{q,d} := \sum_{i \in [|E_q|]} \max_{j \in [|E_d|]} E_{q_i} \cdot E_{d_j}^T$$

- ✓ Independent Encoding
- ✓ Fine-grained Vectors
- ✓ Sub-linear *top-k* search

**How??**

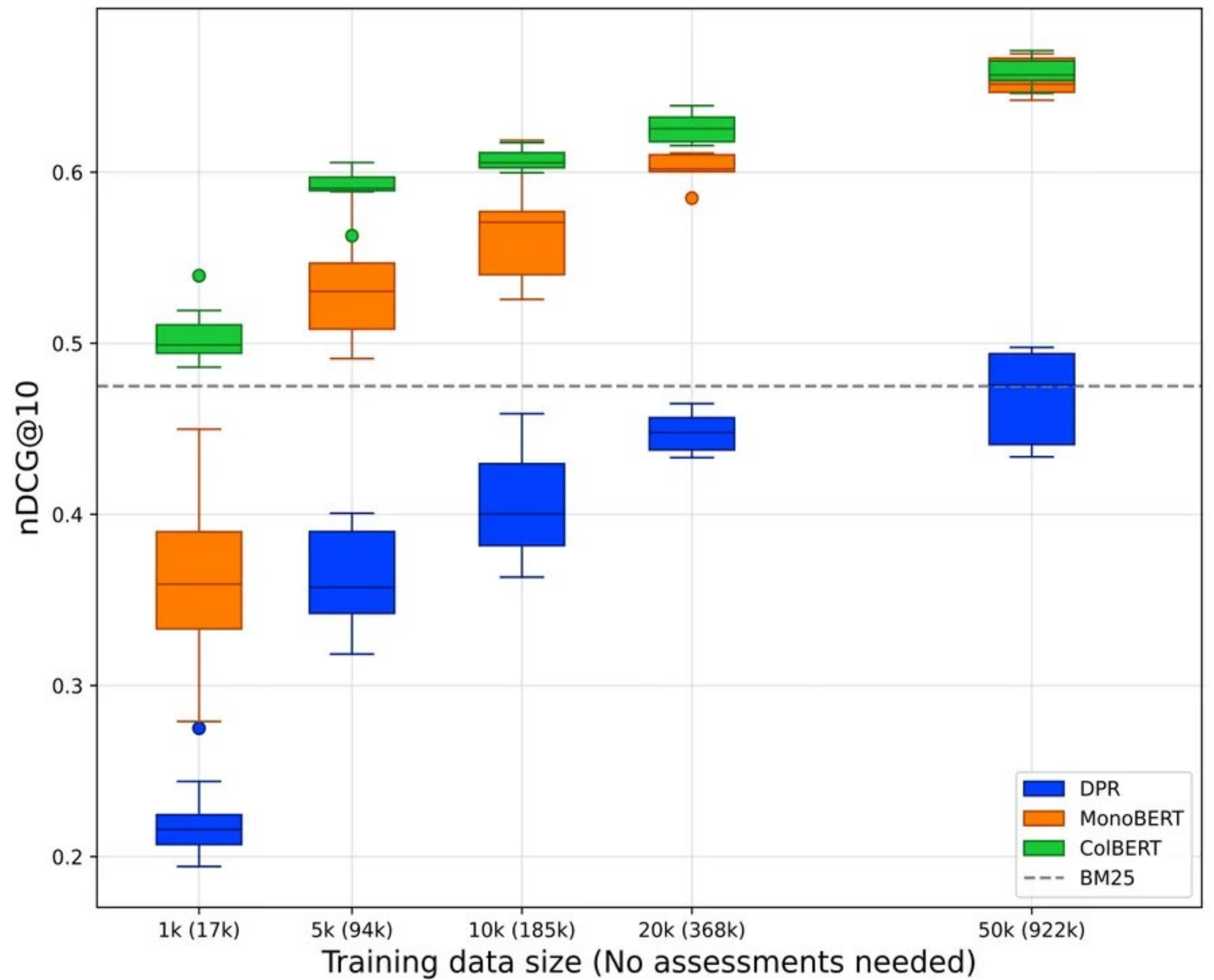
*(Bonus: decompose into parallel IVF queries for  $O(\sqrt{N})$  ANN latency!)*

# What are the effects of these inductive biases?

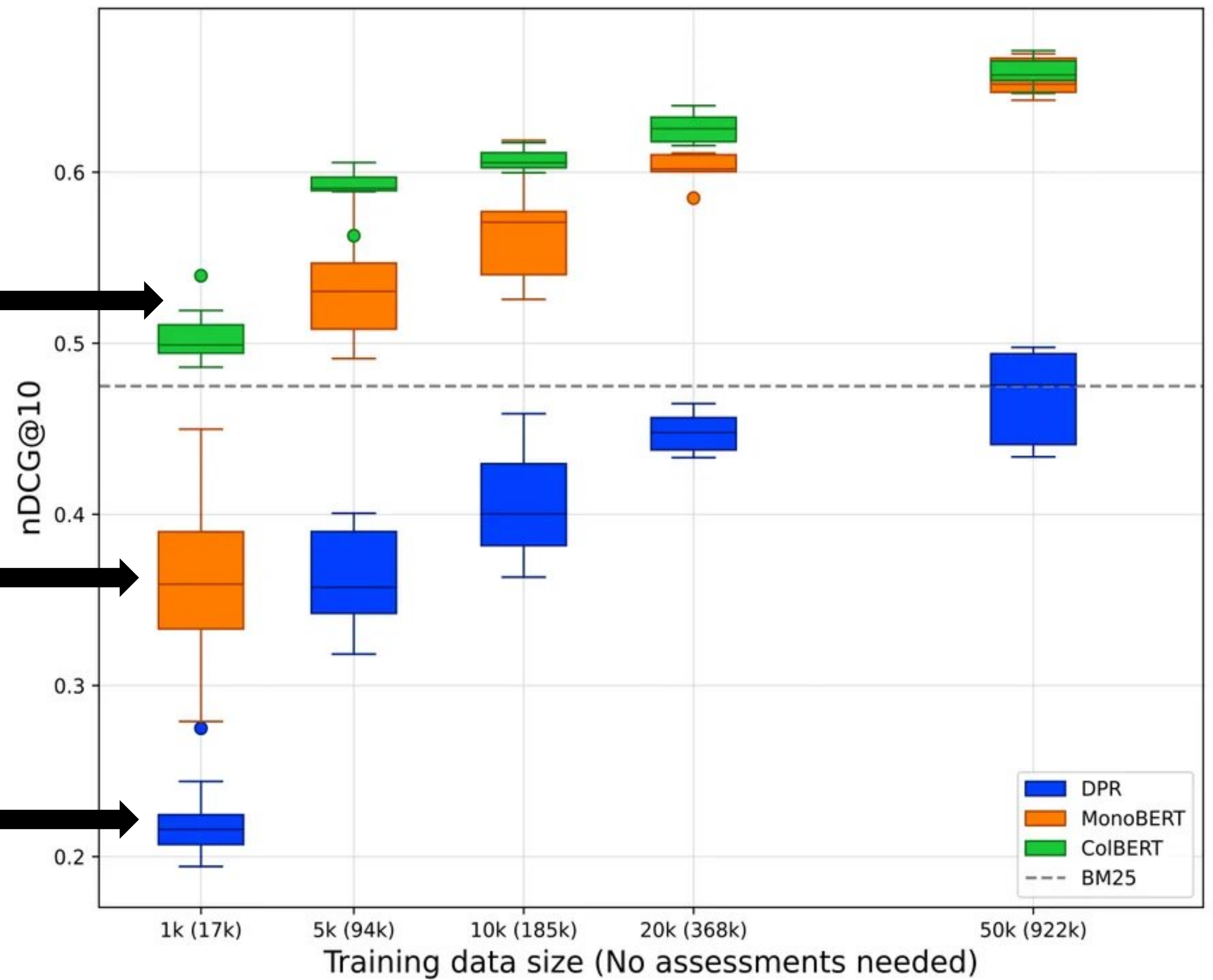
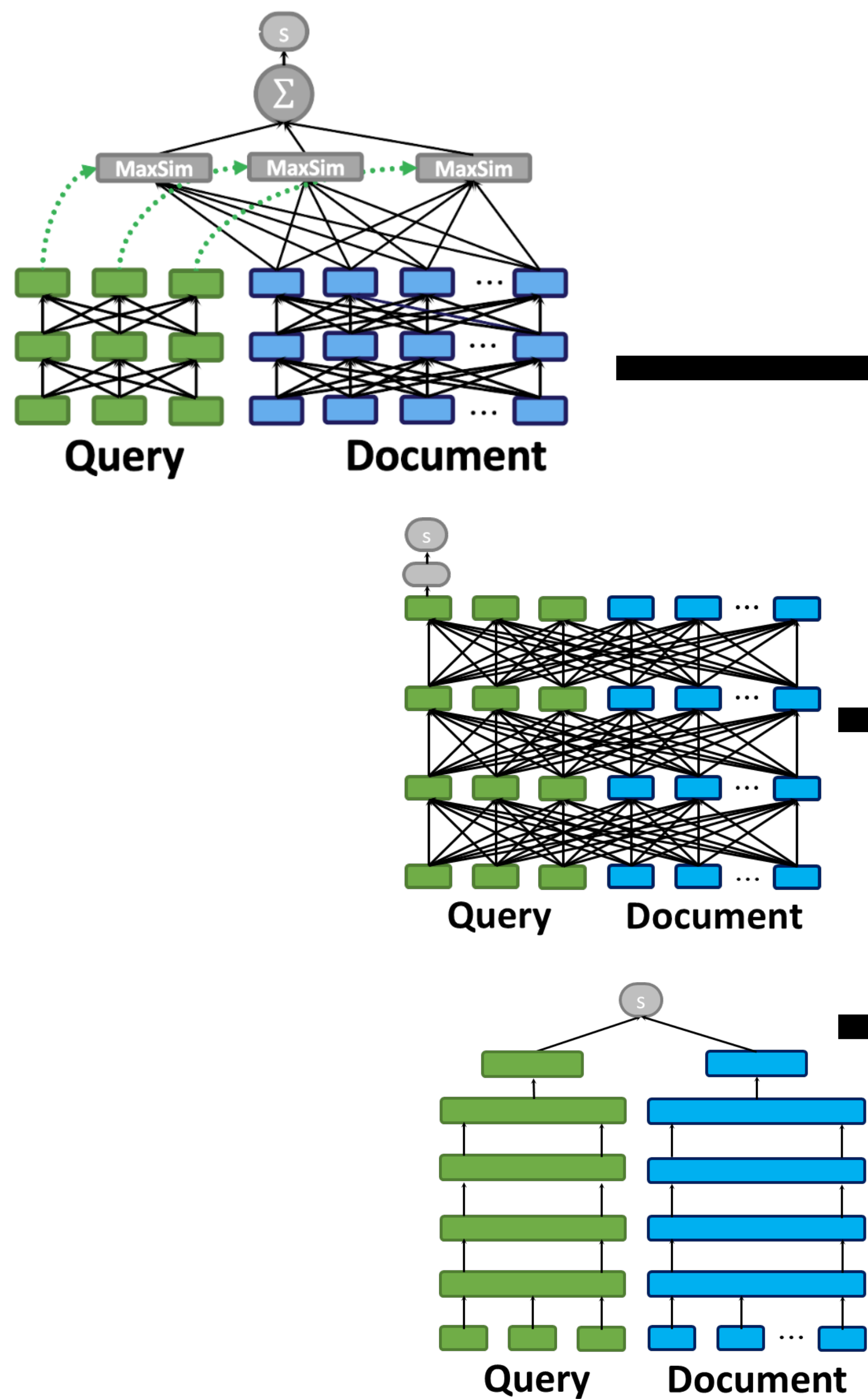
**A few years later, this was perhaps  
tested best by Althammer et al. '23**

(in a fairly unrelated paper on active learning!)

Higher Quality ↑



→ More Data (Expensive)



# [Bonus] Why do the inductive biases have these effects

1. Contextually encoding each token separately is much easier than capturing *all* possible questions about a long document [and nothing else!] in a single vector?
2. Bias from pre-training: BERT's [MASK] tokens build good token representations?
3. Gradient updates at the token level (via MaxSim) generalize well across documents?
4. Cross-Encoders learn more slowly than late interaction. With 1k queries, co-representation and scoring are entangled (i.e., it's too expressive)? What will work better as we scale?
5. On latency: What makes it possible in practice to do late interaction in *sub-linear* time, e.g.  $O(\sqrt{N})$ , by just clustering tokens across all documents?

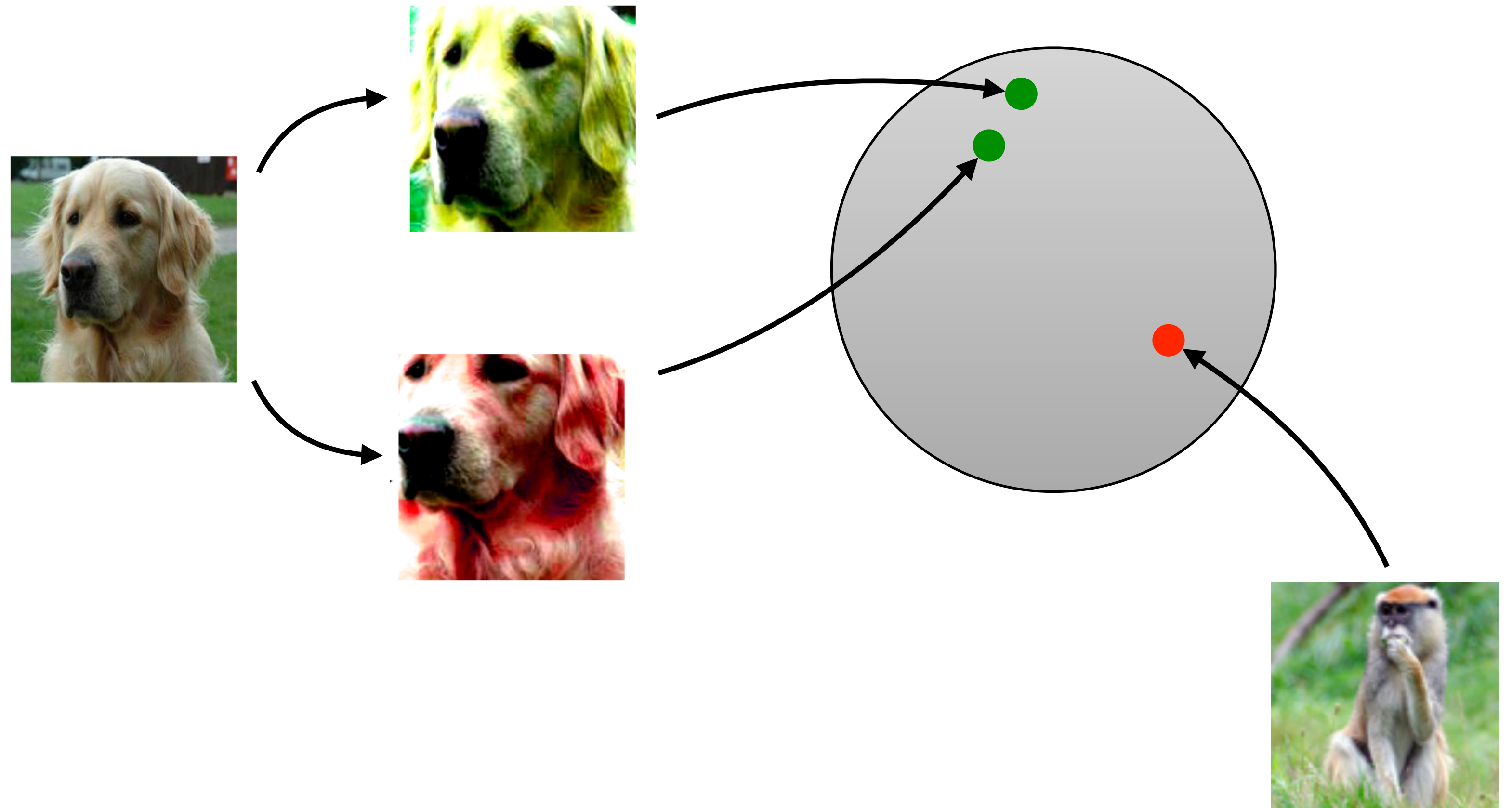
# Limitations of the late interaction paradigm?

1. Requires specialized infra. Constant factors that need care!
  - For instance, how do you store  $>100x$  more vectors? They're extremely compressible, but you have to handle that.
2. Overly narrow inductive bias? Is a sum of maximums always appropriate?
  - Surprisingly hasn't been an issue. But it's an interesting problem: better late-interaction scoring functions that *ideally* remain sub-linear, e.g. see Learned Late Interactions (richer but linear).
3. Less versatile: only works for explicit retrieval tasks. Can't *easily* do other representation learning things like clustering or even symmetric retrieval!
4. With enough training at very large scale, and enough tricks, and much more diverse training sets, a single-vector dual-encoder can often catch up *in-domain* (though this is apples vs. oranges).

# Summary: Retrieval → AI Systems via deep learning...

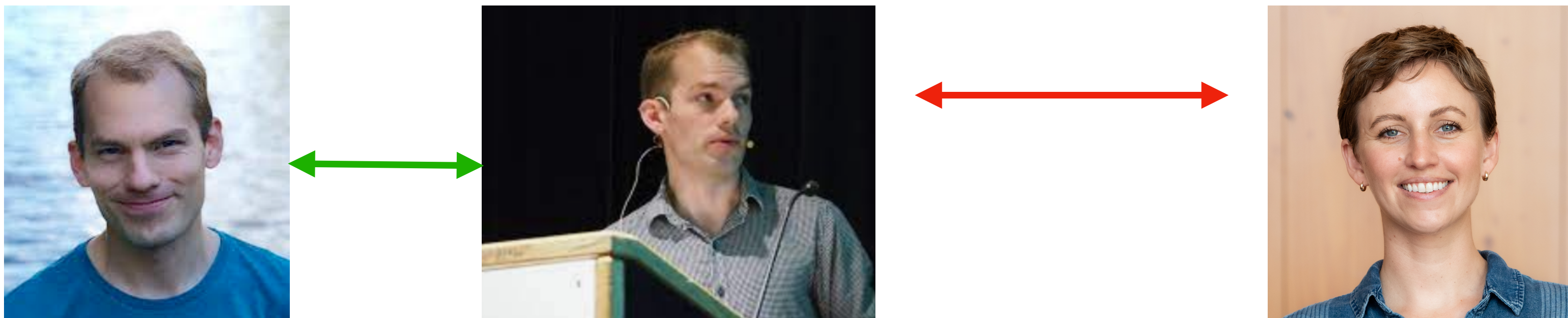
1. IR and Recommendation more or less started a flavor of deep learning I really like. Involves thinking about a user-facing system *end-to-end* and considering *many explicit tradeoffs* when co-designing all parts of the system with the DNN architecture! (More of this flavor in Lectures 15 & 23.)
2. Inductive biases as a DL/Systems co-design tool! Today, we kept doing *decomposition*:
  1. Across Documents  $\Rightarrow$  pointwise scoring! And pointwise or contrastive learning!
  2. Between encoding the Query vs. Document  $\Rightarrow$  dual-encoders.
  3. Across Tokens within a Document (or within a Query)  $\Rightarrow$  late interaction.
3. Many constant factors with very large headroom for systems optimization, meaning that you can get very clever with algorithms & data structures: repeated 100x speed-ups are not unheard of!

# Bonus: You can also have *self-supervised* similarity-based representation learning



# Metric Learning

- Euclidean distance in **input space** may be not ideal
- Instead: learn a metric that respects desired properties
- Goal: learn a metric where:
  - data points that “belong together” are *similar* (close together)
  - data points that are “different” are *dissimilar* (far apart)
- “Supervision”: similarity information.

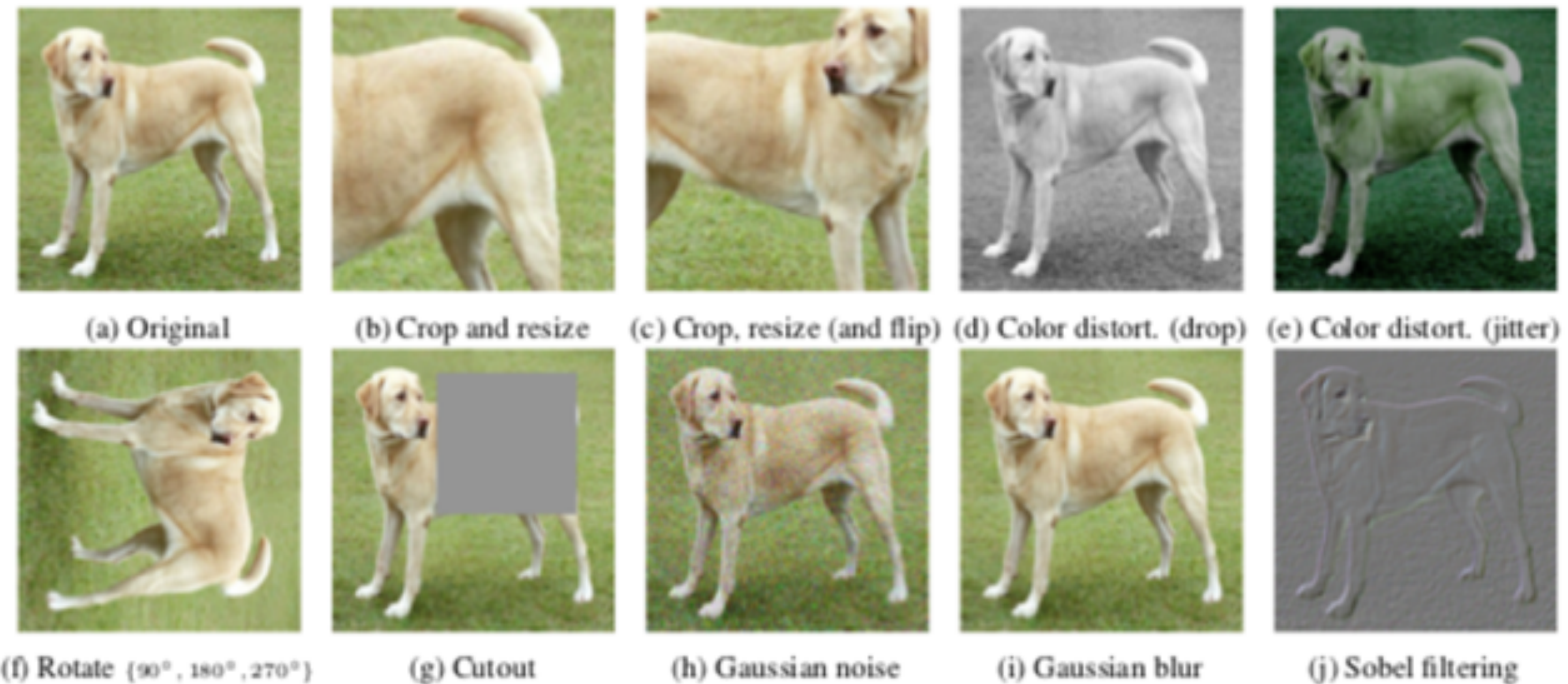


# What are positive and negative examples?

**Negative examples:**  
randomly uniformly  
drawn from data



**Positive examples:**  
perturbations that keep  
semantic meaning,  
data augmentation

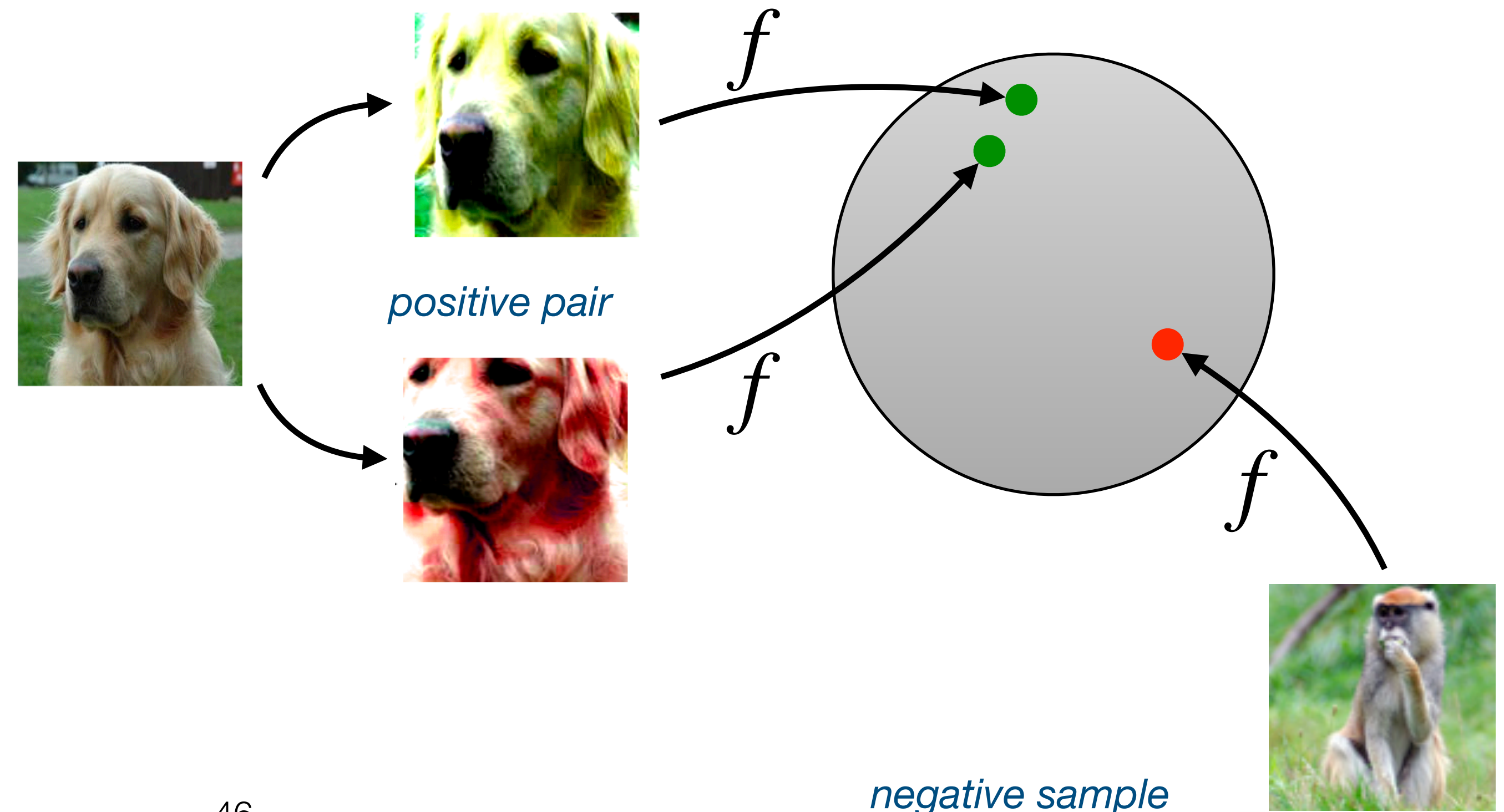


*(Chen, Kornblith, Norouzi, Hinton 2020)*

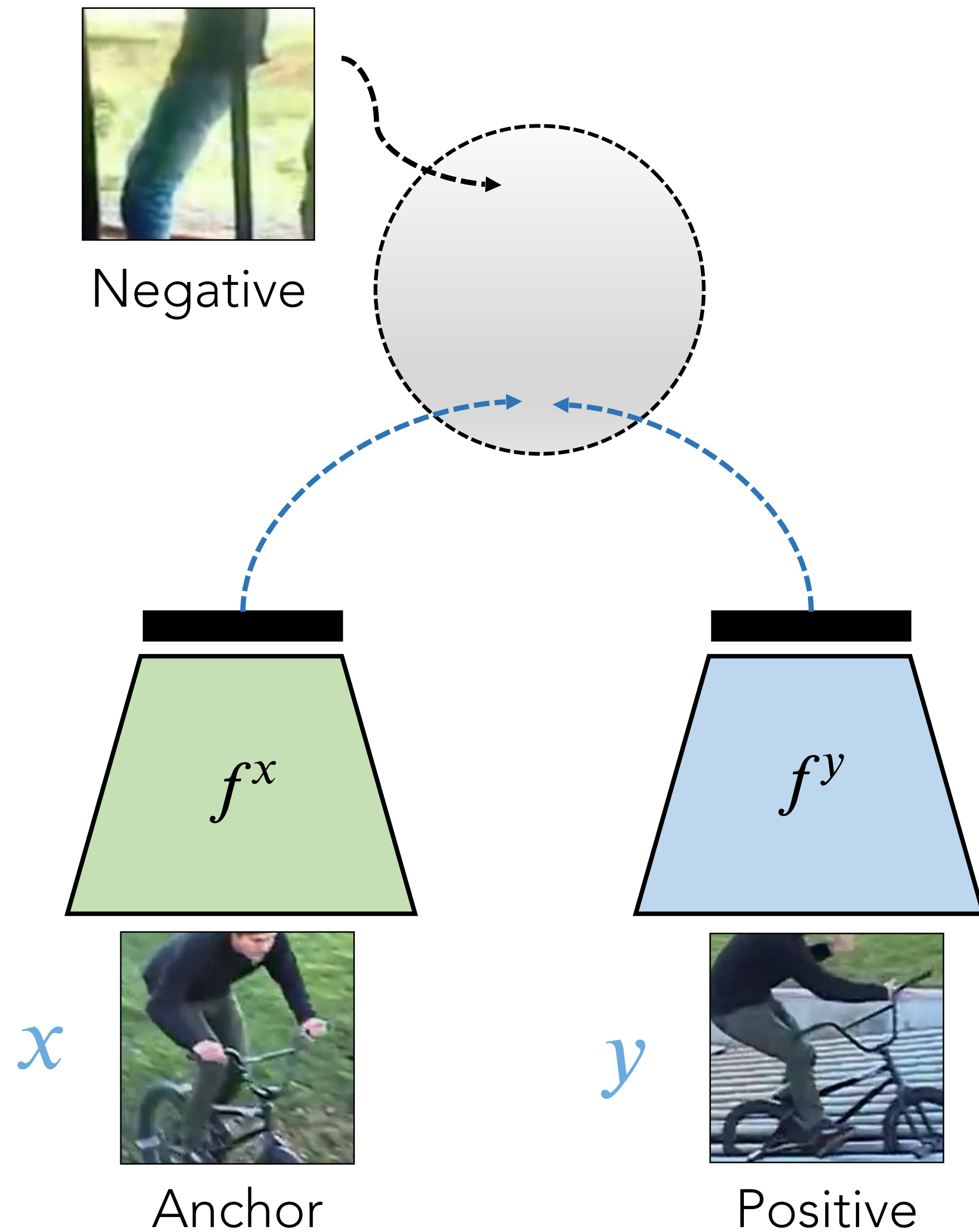
# Positive and negative samples

e.g. SimCLR:

- for each data point in the batch, generate 2 random augmentations as positive pair
- all other  $2(B-1)$  augmented samples in the batch (of size  $B$ ) are used as negatives



# Variations



$(x, y)$  are two “views” of the same scene

## Video Representation Learning

[“Slow Feature Learning”, Wiskott & Sejnowski 2002]

[Mobahi, Collobert, Weston 2009]

[Wang & Gupta 2015]

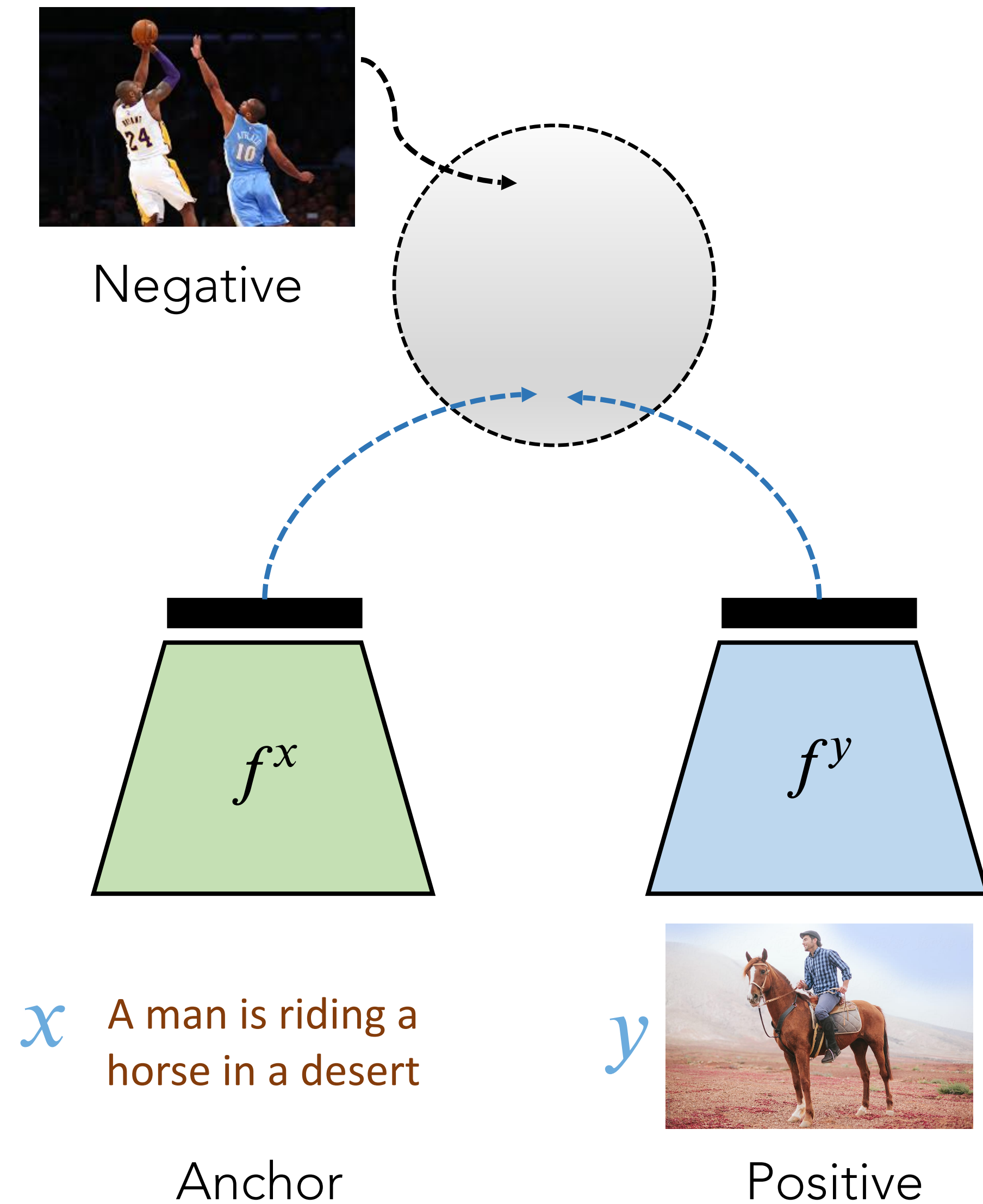
[Isola, Zoran, Krishnan, Adelson 2016]

[Sermanet, Lynch, Chebotar et al. 2018]

[van den Oord, Li, Vinyals 2018]

⋮

# Variations



$(x, y)$  are two “views” of the same scene

Language-Vision Representation Learning

[Karpathy, Joulin, Fei-Fei 2014]

⋮

[CLIP, Radford, Kim et al. 2021]

# What are we “teaching” the model via choice of pairs?

- positive pairs = augmentations of the same data point should be close
- => learned representation is invariant to perturbations induced by data augmentations: **learned invariance**
- Finding the “right” invariances can be challenging for different types of data
- Learned versus hard-coded invariances (geometric DL lecture): when would we use which?